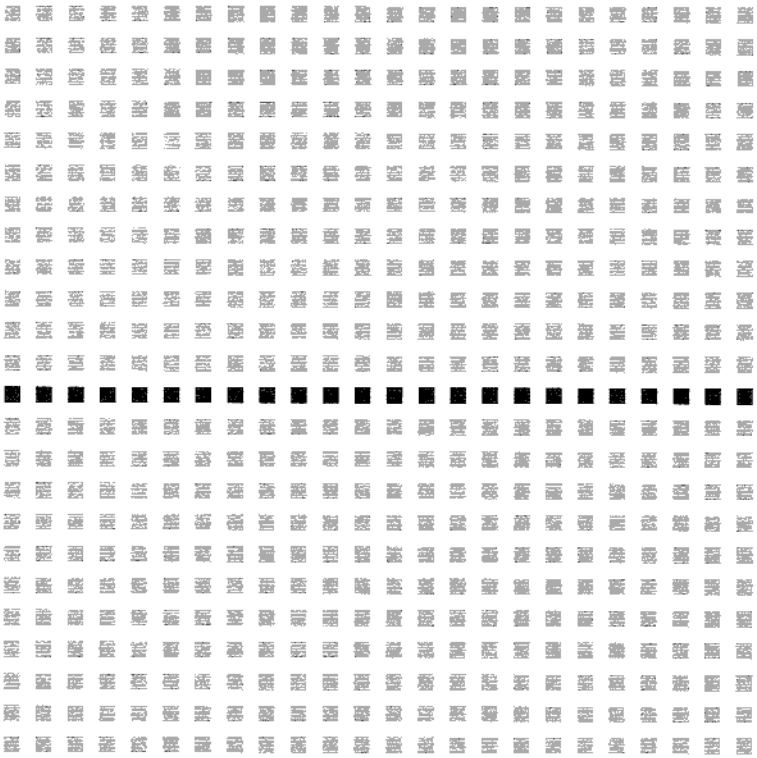
PARTE UM



AUTÔMATOS E LINGUAGENS

1

LINGUAGENS REGULARES

A teoria da computação começa com uma pergunta: O que é um computador? É, talvez, uma questão tola, pois todos sabem que essa coisa sobre a qual estou teclando é um computador. Porém, esses computadores reais são bastante complicados — demasiadamente complicados para nos permitir construir uma teoria matemática manejável sobre eles diretamente. Em vez disso, usamos um computador idealizado chamado um ***modelo computacional***. Como qualquer modelo em ciência, um modelo computacional pode ser preciso em alguns detalhes, mas talvez não em outros. Assim, utilizaremos vários modelos computacionais diferentes, dependendo das características que desejemos focalizar. Começaremos com o modelo mais simples, denominado ***máquina de estados finitos*** ou ***autômato finito***.

1.1 AUTÔMATOS FINITOS

Os autômatos finitos são bons modelos para computadores com uma quantidade extremamente limitada de memória. O que um computador pode fazer com uma memória tão pequena? Muitas coisas úteis! Na verdade, interagimos com esses computadores o tempo todo, pois eles residem no coração de vários dispositivos eletromecânicos.

O controlador de uma porta automática é um exemplo de dispositivo desse tipo. Freqüentemente encontradas em entradas e saídas de supermercados, as portas automáticas abrem-se ao sentir que uma pessoa está se aproximando. Uma porta automática tem um tapete na frente para detectar a presença de uma pessoa que está prestes a atravessar a passagem. Outro tapete está localizado atrás da passagem de modo que o controlador possa manter a porta aberta tempo suficiente para a pessoa passar completamente e também para que a porta não atinja alguém que esteja atrás dela no momento que ela se abre. Essa configuração está mostrada na figura a seguir.

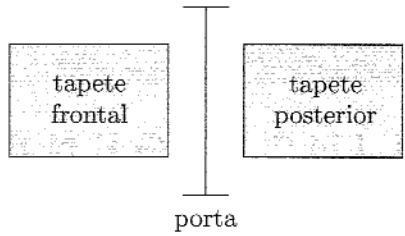


FIGURA 1.1
Visão superior de uma porta automática.

O controlador encontra-se em um dos dois estados: "ABERTO" ou "FECHADO," representando a condição correspondente da porta. Como mostrado nas figuras seguintes, há quatro condições de entrada possíveis: "FRENTE" (significando que uma pessoa está pisando no tapete frontal da porta de passagem), "ATRÁS" (significando que uma pessoa está pisando sobre o tapete posterior da porta de passagem), "AMBOS" (significando que pessoas estão pisando em ambos os tapetes), e "NENHUM" (significando que ninguém está pisando sobre qualquer um dos tapetes).

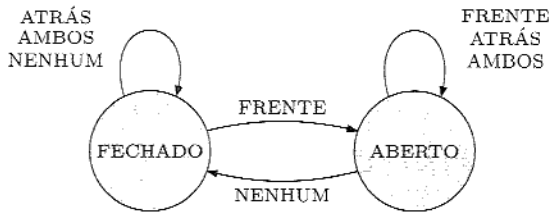


FIGURA 1.2
Diagrama de estados para um controlador de porta automática.

		sinal de entrada			
		NENHUM	FRENTE	ATRÁS	AMBOS
estado	FECHADO	FECHADO	ABERTO	FECHADO	FECHADO
	ABERTO	FECHADO	ABERTO	ABERTO	ABERTO

FIGURA 1.3
Tabela de transição de estados para o controlador de porta automática.

O controlador se move de estado para estado, dependendo da entrada que ele recebe. Estando no estado FECHADO e recebendo como entrada NENHUM ou ATRÁS, ele permanece no estado FECHADO. Adicionalmente, se a entrada AMBOS for recebida, ele permanece em FECHADO porque se a porta for aberta ela pode bater em alguém sobre o tapete posterior. Mas se a entrada FRENTE chegar, ele se move para o estado ABERTO. No estado ABERTO, se a entrada FRENTE, ATRÁS ou AMBOS for recebida, ele permanecerá em ABERTO. Se a entrada NENHUM chegar, ele retornará ao estado FECHADO.

Por exemplo, um controlador poderia iniciar no estado FECHADO e receber a série de sinais de entrada: FRENTE, ATRÁS, NENHUM, FRENTE, AMBOS, NENHUM, ATRÁS e NENHUM. Então ele passaria pela série de estados: FECHADO (iniciando), ABERTO, ABERTO, FECHADO, ABERTO, ABERTO, FECHADO, FECHADO e FECHADO.

Pensar em um controlador de porta automática como um autômato finito é útil porque sugere formas padrão de representação como nas Figuras 1.2 e 1.3. Esse controlador é um computador que tem apenas um bit de memória, capaz de registrar em qual dos dois estados o controlador está. Outros dispositivos comuns têm controladores com memórias bem maiores. Em um controlador de elevador um estado pode representar o andar no qual o elevador está e as entradas podem ser os sinais recebidos dos botões. Esse computador precisaria de vários bits para guardar essa informação. Controladores para diversos aparelhos domésticos, como lavadora de pratos e termostatos eletrônicos, assim como peças de relógios digitais e calculadoras, são exemplos adicionais de computadores com memórias limitadas. O desenho desses dispositivos requer que se tenha em mente a metodologia e a terminologia de autômatos finitos.

Os autômatos finitos e suas contrapartidas probabilísticas ***cadeias de Markov*** são ferramentas úteis quando estamos tentando reconhecer padrões em dados. Esses dispositivos são utilizados em processamento de voz e em reconhecimento de caracteres ópticos. As cadeias de Markov têm sido usadas para modelar e fazer previsões de mudança de preços em mercados financeiros.

Vamos agora dar uma olhada mais cuidadosa em autômatos finitos de uma perspectiva matemática. Desenvolveremos uma definição precisa de autômato finito, uma terminologia para descrever e manipular autômatos finitos e resultados teóricos que descrevem seu poder e suas limitações. Além de lhe dar um entendimento mais claro do que são autômatos finitos e o que eles podem e não podem fazer, esse desenvolvimento teórico vai lhe permitir praticar e se sentir

mais confortável com definições matemáticas, teoremas e provas em um cenário relativamente simples.

Ao começar a descrever a teoria matemática de autômatos finitos, fazemos isso no nível abstrato, sem referência a qualquer aplicação específica. A seguinte figura mostra um autômato finito chamado M_1.

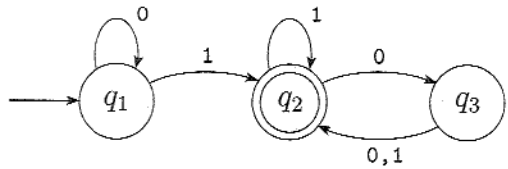


FIGURA 1.4
Um autômato finito chamado M_1 que tem três estados.

A Figura 1.4 é denominada ***diagrama de estado*** de M_1. O autômato tem três ***estados***, rotulados q_1, q_2 e q_3. O ***estado inicial***, q_1, é indicado pela seta apontando para ele a partir do nada. O ***estado de aceitação***, q_2, é aquele com um círculo duplo. As setas saindo de um estado para outro são chamadas ***transições***.

Quando esse autômato recebe uma cadeia de entrada tal como 1101, ele processa essa cadeia e produz uma saída. A saída será ***aceita*** ou ***rejeita***. Consideraremos apenas esse tipo de saída sim/não, no momento, para manter as coisas simples. O processamento começa no estado inicial de M_1. O autômato recebe os símbolos da cadeia de entrada um por um da esquerda para a direita. Após ler cada símbolo, M_1 move-se de um estado para outro ao longo da transição que tem aquele símbolo como seu rótulo. Quando lê o último símbolo, M_1 produz sua saída. A saída será *aceita* se M_1 estiver no estado de aceitação e *rejeita* se não estiver.

Por exemplo, quando alimentamos a cadeia de entrada 1101 à máquina M_1 da Figura 1.4, o processamento procede da seguinte forma.

1. Começa no estado q_1.
2. Lê 1, segue a transição de q_1 para q_2.
3. Lê 1, segue a transição de q_2 para q_2.
4. Lê 0, segue a transição de q_2 para q_3.
5. Lê 1, segue a transição de q_3 para q_2.
6. *Aceita* porque M_1 está no estado de aceitação q_2 no final da entrada.

A experimentação com essa máquina sobre uma variedade de cadeias de entrada revela que ela aceita as cadeias 1, 01, 11 e 0101010101. Na realidade, M_1 aceita qualquer cadeia que termine com o símbolo 1, pois ela vai para seu estado de aceitação q_2 sempre que ela lê o símbolo 1. Além disso, ela aceita as cadeias 100, 0100, 110000 e 0101000000, e qualquer cadeia que termine com

um número par de 0s seguindo o último 1. Ela rejeita outras cadeias, como 0, 10, 101000. Você pode descrever a linguagem constituída de todas as cadeias que M_1 aceita? Faremos isso em breve.

DEFINIÇÃO FORMAL DE UM AUTÔMATO FINITO

Na seção precedente usamos diagramas de estado para introduzir os autômatos finitos. Agora, definimos autômatos finitos formalmente. Embora os diagramas de estado sejam mais fáceis de entender intuitivamente, necessitamos da definição formal também, por duas razões específicas.

Primeiro, uma definição formal é precisa. Ela resolve quaisquer incertezas sobre o que é permitido em um autômato finito. Se estivesse em dúvida sobre se autômatos finitos podem ou não ter 0 estados de aceitação ou se eles devem ter exatamente uma transição saindo de cada estado para cada símbolo de entrada possível, então você poderia consultar a definição formal e verificar que a resposta é sim em ambos os casos. Segundo, uma definição formal provê notação. Uma boa notação o ajuda a pensar e expressar seus pensamentos claramente.

A linguagem de uma definição formal é um tanto misteriosa, tendo alguma semelhança com a linguagem de um documento legal. Ambos necessitam ser precisos, e todo detalhe deve ser explicitado.

Um autômato finito tem várias partes. Possui um conjunto de estados e regras para ir de um estado para outro, dependendo do símbolo de entrada. Possui um alfabeto de entrada que indica os símbolos de entrada permitidos. Tem um estado inicial e um conjunto de estados de aceitação. A definição formal diz que um autômato finito é uma lista daqueles cinco objetos: conjunto de estados, alfabeto de entrada, regras para movimentação, estado inicial e estados de aceitação. Em linguagem matemática, uma lista de cinco elementos é freqüentemente chamada 5-upla. Daí, definimos um autômato finito como sendo uma 5-upla consistindo dessas cinco partes.

Usamos algo denominado ***função de transição***, geralmente denotado por δ, para definir as regras de movimentação. Se o autômato finito tem uma seta de um estado x para um estado y rotulada com o símbolo de entrada 1, isso significa que, se o autômato está no estado x quando lê 1, então ele se move para o estado y. Podemos indicar a mesma coisa com a função de transição dizendo que $\delta(x, 1) = y$. Essa notação é uma espécie de abreviação matemática. Colocando tudo junto, chegamos à definição formal de autômatos finitos.

DEFINIÇÃO 1.5

Um ***autômato finito*** é uma 5-upla $(Q, \Sigma, \delta, q_0, F)$, onde

1. Q é um conjunto finito conhecido como os ***estados***,
2. Σ é um conjunto finito chamado o ***alfabeto***,
3. $\delta\colon Q \times \Sigma \longrightarrow Q$ é a ***função de transição***,[1]
4. $q_0 \in Q$ é o ***estado inicial***, e
5. $F \subseteq Q$ é o ***conjunto de estados de aceitação***.[2]

A definição formal descreve precisamente o que queremos dizer por autômato finito. Por exemplo, retornando à pergunta anterior sobre se 0 estados de aceitação é permissível, você pode ver que fazer F ser o conjunto vazio $\emptyset$ resulta em 0 estados de aceitação, o que é permissível. Além disso, a função de transição δ especifica exatamente um próximo estado para cada combinação possível de um estado e um símbolo de entrada. Isso responde à nossa outra pergunta afirmativamente, mostrando que exatamente uma seta de transição sai de cada estado para cada símbolo de entrada possível.

Podemos utilizar a notação da definição formal para descrever um autômato finito especificando cada uma das cinco partes listadas na Definição 1.5. Por exemplo, vamos retornar ao autômato finito M_1 que discutimos anteriormente, redesenhado aqui por conveniência.

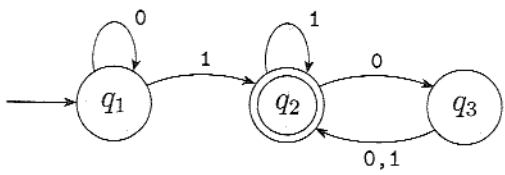


FIGURA 1.6
O autômato finito M_1.

Podemos descrever M_1 formalmente escrevendo $M_1 = (Q, \Sigma, \delta, q_1, F)$, onde

1. $Q = \{q_1, q_2, q_3\}$,
2. $\Sigma = \{0,1\}$,

[1] Remeta-se à página 7 se você estiver incerto sobre o significado de $\delta\colon Q \times \Sigma \longrightarrow Q$.
[2] Os estados de aceitação às vezes são chamados ***estados finais***.

3. δ é descrita como

	0	1
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_2	q_2

4. q_1 é o estado inicial, e
5. $F = \{q_2\}$.

Se A é o conjunto de todas as cadeias que a máquina M aceita, dizemos que A é a ***linguagem da máquina M*** e escrevemos $L(M) = A$. Dizemos que ***M reconhece A*** ou que ***M aceita A***. Em razão do termo *aceita* ter significados diferentes quando nos referimos a máquinas que aceitam cadeias e a máquinas que aceitam linguagens, preferimos o termo *reconhece* para linguagens de forma a evitar confusão.

Uma máquina pode aceitar várias cadeias, mas ela sempre reconhece uma única linguagem. Se a máquina não aceitar nenhuma cadeia, ela ainda assim reconhecerá uma linguagem — a saber, a linguagem vazia $\emptyset$.

Em nosso exemplo, seja

$$A = \{w \mid w \text{ contém pelo menos um } 1 \text{ e um número par de } 0\text{s segue o último } 1\}.$$

Então $L(M_1) = A$, ou, equivalentemente, M_1 reconhece A.

EXEMPLOS DE AUTÔMATOS FINITOS

EXEMPLO 1.7

Segue o diagrama de estados do autômato finito M_2.

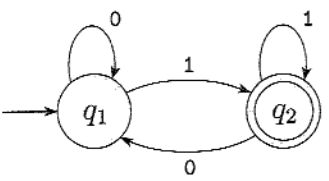


FIGURA 1.8
Diagrama de estados do autômato finito de dois estados M_2.

Na descrição formal $M_2 = (\{q_1, q_2\}, \{0,1\}, \delta, q_1, \{q_2\})$. A função de transição δ é

	0	1
q_1	q_1	q_2
q_2	q_1	q_2

Lembre-se que o diagrama de estados de M_2 e a descrição formal de M_2 contêm a mesma informação, só que de forma diferente. Você pode sempre ir de um para o outro se necessário.

Uma boa maneira para começar a entender uma máquina é testá-la com algumas amostras de cadeias de entrada. Quando você faz esses "experimentos" para ver como a máquina funciona, o seu método de funcionamento freqüentemente se torna aparente. Para a cadeia 1101, a máquina M_2 começa no estado inicial q_1 e procede primeiro para o estado q_2 após ler o primeiro 1, e então para os estados q_2, q_1 e q_2 após ler 1, 0 e 1. A cadeia é aceita porque q_2 é um estado de aceitação. Mas a cadeia 110 deixa M_2 no estado q_1; portanto, ela é rejeitada. Depois de tentar uns poucos exemplos mais, você veria que M_2 aceita todas as cadeias que terminam com 1. Conseqüentemente, $L(M_2) = \{w \mid w \text{ termina com um } 1\}$.

EXEMPLO 1.9

Considere o autômato finito M_3.

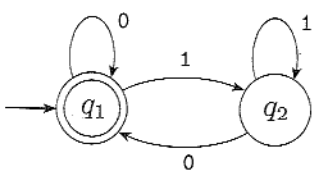


FIGURA 1.10
Diagrama de estados do autômato finito de dois estados M_3.

A máquina M_3 é semelhante à M_2 exceto pela localização do estado de aceitação. Como de costume, a máquina aceita todas as cadeias que a deixam num estado de aceitação quando ela tiver terminado de ler. Note que, em razão do estado inicial também ser um estado de aceitação, M_3 aceita a cadeia vazia ε. Assim que a máquina começa a ler a cadeia vazia ela está no final, portanto, se o estado inicial é um estado de aceitação, ε é aceita. Além da cadeia vazia, essa máquina aceita qualquer cadeia terminando com um 0. Aqui,

$$L(M_3) = \{w \mid w \text{ é a cadeia vazia } \varepsilon \text{ ou termina em um } 0\}.$$

EXEMPLO 1.11

A figura seguinte mostra uma máquina de cinco estados M_4.

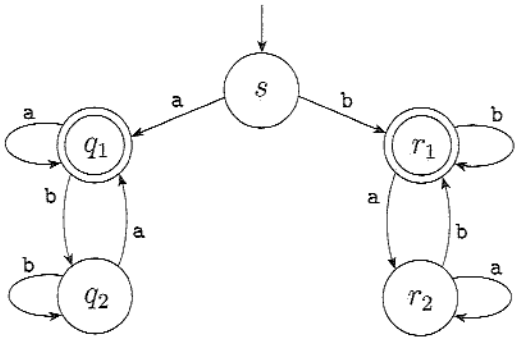


FIGURA 1.12
Autômato finito M_4.

A máquina M_4 tem dois estados de aceitação, q_1 e r_1, e opera sobre o alfabeto $\Sigma = \{\mathtt{a}, \mathtt{b}\}$. Um pouco de experimentação nos mostra que ela aceita as cadeias a, b, aa, bb e bab, mas não as cadeias ab, ba ou bbba. Essa máquina começa no estado s, e depois que ela lê o primeiro símbolo na entrada, ela vai ou para a esquerda nos estados q ou para a direita nos estados r. Em ambos os casos ela nunca pode retornar ao estado inicial (diferentemente dos exemplos anteriores), pois ela não tem como sair de qualquer outro estado e voltar para s. Se o primeiro símbolo na cadeia de entrada for a, então ela vai para a esquerda e aceita quando a cadeia termina com um a. Similarmente, se o primeiro símbolo for b, a máquina vai para a direita e aceita quando a cadeia termina em um b. Portanto M_4 aceita todas as cadeias que começam e terminam com a ou que começam e terminam com b. Em outras palavras, M_4 aceita cadeias que começam e terminam com o mesmo símbolo. ▫

EXEMPLO 1.13

A Figura 1.14 mostra a máquina M_5, que tem um alfabeto de entrada de quatro símbolos, $\Sigma = \{\langle\text{RESET}\rangle, 0, 1, 2\}$. Tratamos $\langle\text{RESET}\rangle$ como um único símbolo.

A máquina M_5 mantém um contador da soma dos símbolos numéricos de entrada que ela lê, módulo 3. Toda vez que recebe o símbolo $\langle\text{RESET}\rangle$ ela reinicia o contador para 0. Ela aceita se a soma for 0, módulo 3, ou, em outras palavras, se a soma for um múltiplo de 3. ▫

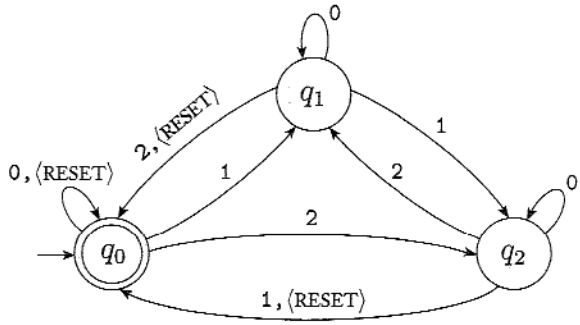


FIGURA 1.14
Autômato finito M_5.

Em alguns casos não é possível descrever um autômato finito por diagrama de estados. Isso pode ocorrer quando o diagrama é demasiado grande para se desenhar ou se, como no exemplo a seguir, a descrição depende de algum parâmetro não especificado. Nesses casos recorremos a uma descrição formal para especificar a máquina.

EXEMPLO 1.15

Considere uma generalização do Exemplo 1.13, usando o mesmo alfabeto de quatro símbolos Σ. Para cada $i \geq 1$ seja A_i a linguagem de todas as cadeias em que a soma dos números é um múltiplo de i, exceto que a soma é reinicializada para 0 sempre que o símbolo ⟨RESET⟩ aparece. Para cada A_i damos um autômato finito B_i, reconhecendo A_i. Descrevemos a máquina B_i formalmente da seguinte forma: $B_i = (Q_i, \Sigma, \delta_i, q_0, \{q_0\})$, onde Q_i é o conjunto de i estados $\{q_0, q_1, q_2, \ldots, q_{i-1}\}$, e desenhamos a função de transição δ_i de modo que para cada j, se B_i está em q_j, a soma corrente é j, módulo i. Para cada q_j faça

$$\delta_i(q_j, 0) = q_j,$$
$$\delta_i(q_j, 1) = q_k, \text{ onde } k = j + 1 \text{ módulo } i,$$
$$\delta_i(q_j, 2) = q_k, \text{ onde } k = j + 2 \text{ módulo } i, \text{ e}$$
$$\delta_i(q_j, \langle\text{RESET}\rangle) = q_0.$$

DEFINIÇÃO FORMAL DE COMPUTAÇÃO

Até agora descrevemos autômatos finitos informalmente, usando diagramas de estados, e com uma definição formal, como uma 5-upla. A descrição informal é mais fácil de absorver inicialmente, mas a definição formal é útil para tornar a

noção precisa, resolvendo quaisquer ambigüidades que possam ter ocorrido na descrição informal. A seguir fazemos o mesmo para uma computação de um autômato finito. Já temos uma idéia informal da maneira pela qual ele computa, e agora a formalizamos matematicamente.

Seja $M = (Q, \Sigma, \delta, q_0, F)$ um autômato finito e suponha que $w = w_1 w_2 \cdots w_n$ seja uma cadeia onde cada w_i é um membro do alfabeto Σ. Então M ***aceita*** w se existe uma seqüência de estados $r_0, r_1, \ldots, r_n$ em Q com três condições:

1. $r_0 = q_0$,
2. $\delta(r_i, w_{i+1}) = r_{i+1}$, para $i = 0, \ldots, n-1$, e
3. $r_n \in F$.

A condição 1 diz que a máquina começa no estado inicial. A condição 2 diz que a máquina vai de estado para estado conforme a função de transição. A condição 3 diz que a máquina aceita sua entrada se ela termina em um estado de aceitação. Dizemos que M ***reconhece a linguagem*** A se $A = \{w \mid M \text{ aceita } w\}$.

DEFINIÇÃO 1.16

Uma linguagem é chamada de uma ***linguagem regular*** se algum autômato finito a reconhece.

EXEMPLO 1.17

Seja a máquina M_5 do Exemplo 1.13. Seja w a cadeia

$$10\langle\text{RESET}\rangle 22\langle\text{RESET}\rangle 012$$

Então M_5 aceita w conforme a definição formal de computação, porque a seqüência de estados na qual ela entra quando está computando sobre w é

$$q_0, q_1, q_1, q_0, q_2, q_1, q_0, q_0, q_1, q_0,$$

o que satisfaz as três condições. A linguagem de M_5 é

$$L(M_5) = \{w \mid \text{a soma dos símbolos em } w \text{ é 0 módulo 3, exceto que } \langle\text{RESET}\rangle \text{ retorna o contador para 0}\}.$$

Como M_5 reconhece essa linguagem, ela é uma linguagem regular. ■

PROJETANDO AUTÔMATOS FINITOS

Seja um autômato ou uma peça de arte, projetar é um processo criativo. Como tal ele não pode ser reduzido a uma receita ou fórmula simples. Entretanto, você pode achar uma abordagem específica útil ao projetar vários tipos de autômatos. Ou seja, ponha-se *a si próprio* no lugar da máquina que está tentando projetar e então veja como você se conduziria para realizar a tarefa da máquina. Fazer de

conta que você é a máquina é um truque psicológico que ajuda a aplicar toda a sua mente no processo de projetar.

Vamos projetar um autômato finito usando o método "leitor como autômato" que acabamos de descrever. Suponha que lhe seja dada alguma linguagem e você deseje projetar um autômato finito que a reconheça. Fazendo de conta que é o autômato, você recebe uma cadeia de entrada e tem que determinar se ela é um membro da linguagem que se supõe que o autômato reconheça. Você vai vendo os símbolos na cadeia um por um. Depois de cada símbolo você tem de decidir se a cadeia vista até então está na linguagem. A razão é que você, como a máquina, não sabe quando o final da cadeia está vindo, e portanto tem que estar sempre pronto com a resposta.

Primeiro, de modo a tomar essas decisões, você tem que perceber o que precisa lembrar sobre a cadeia na medida em que a está lendo. Por que não simplesmente lembrar de tudo que você viu? Lembre-se que você está fazendo de conta que é um autômato finito e que esse tipo de máquina tem somente um número finito de estados, o que significa uma memória finita. Imagine que a entrada seja extremamente longa — digamos, daqui até a Lua — de modo que você não poderia de forma alguma se lembrar da coisa inteira. Você possui uma memória finita — digamos, uma única folha de papel — que tem uma capacidade de armazenamento limitada. Felizmente, para muitas linguagens você não precisa se lembrar de toda a entrada. Você precisa se lembrar somente de certa informação crucial. Exatamente qual informação é crucial depende da linguagem específica considerada.

Por exemplo, suponha que o alfabeto seja $\{0,1\}$ e que a linguagem consista de todas as cadeias com um número ímpar de 1s. Você deseja construir um autômato finito E_1 para reconhecer essa linguagem. Fazendo de conta que é o autômato, você começa obtendo uma cadeia de entrada de 0s e 1s, símbolo a símbolo. Você precisa lembrar a cadeia inteira vista até então para determinar se o número de 1s é ímpar? É claro que não. Simplesmente lembre se o número de 1s visto até então é par ou ímpar e mantenha essa informação à medida lê novos símbolos. Se você ler um 1, inverta a resposta, mas se você ler um 0, deixe a resposta como está.

Mas como isso lhe ajuda a projetar E_1? Uma vez que tenha determinado a informação necessária para lembrar sobre a cadeia à medida que ela está sendo lida, você representa essa informação como uma lista finita de possibilidades. Nessa instância, as possibilidades seriam

1. par até agora, e
2. ímpar até agora.

Aí então você atribui um estado a cada uma das possibilidades. Estes são os estados de E_1, como mostrado a seguir.

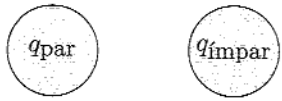


FIGURA 1.18
Os dois estados q_{par} e $q_{ímpar}$.

Em seguida, você atribui as transições vendo como ir de uma possibilidade para outra ao ler um símbolo. Portanto, se o estado q_{par} representa a possibilidade par e o estado $q_{ímpar}$ representa a possibilidade ímpar, você faz as transições trocarem de estado com um 1 e permanecerem como estão com um 0, como mostrado a seguir.

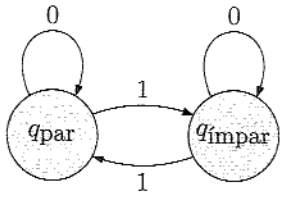


FIGURA 1.19
Transições dizendo como as possibilidades se rearranjam.

Em seguida, você coloca como estado inicial aquele correspondendo à possibilidade associada com ter visto 0 símbolos até então (a cadeia vazia ε). Nesse caso, o estado inicial corresponde ao estado q_{par} porque 0 é um número par. Por último, ponha como estados de aceitação aqueles correspondendo a possibilidades nas quais você deseja aceitar a cadeia de entrada. Faça com que $q_{ímpar}$ seja um estado de aceitação porque você deseja aceitar quando você tiver visto um número ímpar de 1s. Essas adições são mostradas na figura abaixo.

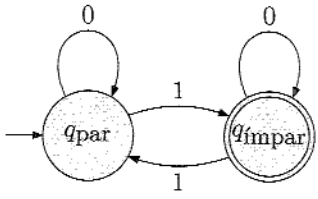


FIGURA 1.20
Adicionando os estados inicial e de aceitação.

EXEMPLO 1.21

Este exemplo mostra como projetar um autômato finito E_2 para reconhecer a linguagem regular de todas as cadeias que contêm a cadeia 001 como uma subcadeia. Por exemplo, 0010, 1001, 001 e 11111110011111 estão todas na linguagem, mas 11 e 0000 não estão. Como você reconheceria essa linguagem se estivesse fazendo de conta ser E_2? À medida que os símbolos chegassem, você inicialmente saltaria sobre todos os 1s. Se você chegar num 0, então nota que pode ter acabado de ver o primeiro dos três símbolos no padrão 001 que você está buscando. Se nesse ponto você vê um 1, houve muito poucos 0s, portanto você volta a saltar sobre 1s. Mas, se vê um 0 nesse ponto, você deve lembrar que acabou de ver dois símbolos do padrão. Agora você simplesmente precisa continuar fazendo uma varredura até que veja um 1. Se você o encontrar, lembre-se de que conseguiu achar o padrão e continue lendo a cadeia de entrada até que você chegue no final.

Portanto, existem quatro possibilidades: você

1. não viu quaisquer símbolos do padrão,
2. acaba de ver um 0,
3. acaba de ver 00, ou
4. acaba de ver o padrão inteiro 001.

Atribua os estados q, q_0, q_{00} e q_{001} a essas possibilidades. Você pode atribuir as transições observando que de q lendo 1 você permanece em q, mas lendo 0 você se move para q_0. Em q_0 lendo 1 você retorna a q, mas lendo 0 você se move para q_{00}. Em q_{00}, lendo 1 você se move para q_{001}, mas lendo 0 você permanece em q_{00}. Finalmente, em q_{001} lendo 0 ou 1 deixa você em q_{001}. O estado inicial é q, e o único estado de aceitação é q_{001}, como mostrado na Figura 1.22.

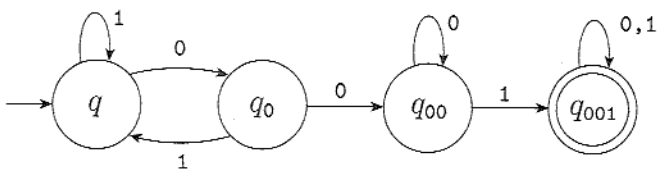


FIGURA 1.22
Aceita cadeias contendo 001.

AS OPERAÇÕES REGULARES

Nas duas seções precedentes introduzimos e definimos autômatos finitos e linguagens regulares. Agora começamos a investigar suas propriedades. Isso vai ajudar a desenvolver uma caixa de ferramentas de técnicas para utilizar quando você projetar autômatos para reconhecer linguagens específicas. A caixa de

ferramentas também incluirá formas de provar que certas linguagens são não-regulares (isto é, além da capacidade de autômatos finitos).

Em aritmética, os objetos básicos são números e as ferramentas são operações para manipulá-los, tais como $+$ e $\times$. Na teoria da computação os objetos são linguagens e as ferramentas incluem operações especificamente projetadas para manipulá-las. Definimos três operações sobre linguagens, chamadas ***operações regulares***, e as usamos para estudar propriedades de linguagens regulares.

DEFINIÇÃO 1.23

Sejam A e B linguagens. Definimos as operações regulares ***união***, ***concatenação*** e ***estrela*** da seguinte forma.

- **União**: $A \cup B = \{x \mid x \in A \text{ ou } x \in B\}$.
- **Concatenação**: $A \circ B = \{xy \mid x \in A \text{ e } y \in B\}$.
- **Estrela**: $A^* = \{x_1 x_2 \ldots x_k \mid k \geq 0 \text{ e cada } x_i \in A\}$.

Você já está familiarizado com a operação de união. Ela simplesmente toma todas as cadeias em ambas A e B e as junta em uma linguagem.

A operação de concatenação é um pouco mais complicada. Ela acrescenta uma cadeia de A na frente de uma cadeia de B de todas as maneiras possíveis para obter as cadeias na nova linguagem.

A operação estrela é um pouco diferente das outras duas porque ela se aplica a uma única linguagem em vez de duas linguagens diferentes. Ou seja, a operação estrela é uma ***operação unária*** ao invés de uma ***operação binária***. Ela funciona justapondo qualquer número de cadeias em A para obter uma cadeia na nova linguagem. Em razão de "qualquer número" incluir 0 como uma possibilidade, a cadeia vazia ε é sempre um membro de A^*, independentemente do que A seja.

EXEMPLO 1.24

Suponha que o alfabeto Σ seja o alfabeto padrão de 26 letras $\{\mathtt{a}, \mathtt{b}, \ldots, \mathtt{z}\}$. Se $A = \{\mathtt{legal}, \mathtt{ruim}\}$ e $B = \{\mathtt{garoto}, \mathtt{garota}\}$, então

$$A \cup B = \{\mathtt{legal}, \mathtt{ruim}, \mathtt{garoto}, \mathtt{garota}\},$$

$$A \circ B = \{\mathtt{legalgaroto}, \mathtt{legalgarota}, \mathtt{ruimgaroto}, \mathtt{ruimgarota}\}, \text{ e}$$

$$A^* = \{\varepsilon, \mathtt{legal}, \mathtt{ruim}, \mathtt{legallegal}, \mathtt{legalruim}, \mathtt{ruimlegal}, \mathtt{ruimruim}, \mathtt{legallegallegal}, \mathtt{legallegalruim}, \mathtt{legalruimlegal}, \mathtt{legalruimruim}, \ldots\}.$$

Seja $\mathcal{N} = \{1, 2, 3, \ldots\}$ o conjunto dos números naturais. Quando dizemos que $\mathcal{N}$ é *fechado sob multiplicação* queremos dizer que, para quaisquer x e y em

$\mathcal{N}$, o produto $x \times y$ também está em $\mathcal{N}$. Diferentemente, $\mathcal{N}$ não é fechado sob divisão, pois 1 e 2 estão em $\mathcal{N}$ mas $1/2$ não está. Em geral, uma coleção de objetos é ***fechada*** sob alguma operação se, aplicando-se essa operação a membros da coleção, recebe-se um objeto ainda na coleção. Mostramos a seguir que a coleção de linguagens regulares é fechada sob as três operações regulares. Na Seção 1.3 mostramos que essas são ferramentas úteis para se manipular linguagens regulares e entender o poder dos autômatos finitos. Começamos com a operação de união.

TEOREMA 1.25

A classe de linguagens regulares é fechada sob a operação de união.

Em outras palavras, se A_1 e A_2 são linguagens regulares, o mesmo acontece com $A_1 \cup A_2$.

IDÉIA DA PROVA Temos as linguagens regulares A_1 e A_2 e desejamos mostrar que $A_1 \cup A_2$ também é regular. Em razão de A_1 e A_2 serem regulares, sabemos que algum autômato finito M_1 reconhece A_1 e algum autômato finito M_2 reconhece A_2. Para provar que $A_1 \cup A_2$ é regular exibimos um autômato finito (chame-o M) que reconhece $A_1 \cup A_2$.

Esta é uma prova por construção. Construímos M a partir de M_1 e M_2. A máquina M tem de aceitar sua entrada exatamente quando M_1 ou M_2 a aceitaria, de modo a reconhecer a linguagem da união. Ela funciona *simulando* ambas, M_1 e M_2, e aceitando se uma das simulações aceita.

Como podemos fazer com que a máquina M simule M_1 e M_2? Talvez ela primeiro simule M_1 sobre a entrada e depois simule M_2 sobre a entrada. Mas temos que ser cuidadosos aqui! Uma vez que os símbolos da entrada tenham sido lidos e usados para simular M_1, não podemos "re-enrolar a fita de entrada" para tentar a simulação de M_2. Precisamos de uma outra abordagem.

Faça de conta que você é M. À medida que os símbolos de entrada chegam um por um, você simula ambas M_1 e M_2 simultaneamente. Dessa maneira somente uma passagem sobre a entrada é necessária. Mas você pode controlar ambas as simulações com memória finita? Tudo do que você precisa guardar é o estado em que cada máquina estaria se ela tivesse lido até esse ponto na entrada. Conseqüentemente, você precisa guardar um par de estados. Quantos pares possíveis existem? Se M_1 tem k_1 estados e M_2 tem k_2 estados, o número de pares de estados, um de M_1 e o outro de M_2, é o produto $k_1 \times k_2$. Esse produto será o número de estados em M, um para cada par. As transições de M vão de par para par, atualizando o estado atual para ambas M_1 e M_2. Os estados de aceitação de M são aqueles pares nos quais ou M_1 ou M_2 está em um estado de aceitação.

PROVA

Suponha que M_1 reconheça A_1, onde $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$, e que
M_2 reconheça A_2, onde $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$.

Construa M para reconhecer $A_1 \cup A_2$, onde $M = (Q, \Sigma, \delta, q_0, F)$.

1. $Q = \{(r_1, r_2) | \, r_1 \in Q_1 \text{ and } r_2 \in Q_2\}$.
   Esse conjunto é o ***produto cartesiano*** dos conjuntos Q_1 e Q_2 e é escrito $Q_1 \times Q_2$. Trata-se do conjunto de todos os pares de estados, sendo o primeiro de Q_1 e o segundo de Q_2.
2. Σ, o alfabeto, é o mesmo em M_1 e M_2. Neste teorema e em todos os teoremas similares subseqüentes, assumimos por simplicidade que ambas M_1 e M_2 têm o mesmo alfabeto de entrada Σ. O teorema permanece verdadeiro se elas tiverem alfabetos diferentes, Σ_1 e Σ_2. Aí então modificaríamos a prova para tornar $\Sigma = \Sigma_1 \cup \Sigma_2$.
3. δ, a função de transição, é definida da seguinte maneira. Para cada $(r_1, r_2) \in Q$ e cada $a \in \Sigma$, faça
   $$\delta\big((r_1, r_2), a\big) = \big(\delta_1(r_1, a), \delta_2(r_2, a)\big).$$
   Logo, δ obtém um estado de M (que na realidade é um par de estados de M_1 e M_2), juntamente com um símbolo de entrada, e retorna o próximo estado de M.
4. q_0 é o par (q_1, q_2).
5. F é o conjunto de pares nos quais um dos membros é um estado de aceitação de M_1 ou M_2. Podemos escrevê-lo como
   $$F = \{(r_1, r_2) | \, r_1 \in F_1 \text{ ou } r_2 \in F_2\}.$$
   Essa expressão é a mesma que $F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$. (Note que ela *não* é a mesma que $F = F_1 \times F_2$. O que isso daria ao invés da união?[3])

Isso conclui a construção do autômato finito M que reconhece a união de A_1 e A_2. Essa construção é bastante simples e, portanto, sua correção é evidente da estratégia descrita na idéia da prova. Construções mais complicadas requerem discussão adicional para provar correção. Uma prova formal de correção para uma construção desse tipo geralmente procede por indução. Para um exemplo de uma construção que é provada ser correta, veja a prova do Teorema 1.54. A maioria das construções que você vai encontrar neste curso são bastante simples e, portanto, não requerem uma prova formal de correção.

Acabamos de mostrar que a união de duas linguagens regulares é regular, provando, assim, que a classe de linguagens regulares é fechada sob a operação de união. Agora nos voltamos para a operação de concatenação e tentamos mostrar que a classe de linguagens regulares é fechada sob essa operação também.

---

[3] Essa expressão definiria os estados de aceitação de M como aqueles para os quais *ambos* os membros do par são estados de aceitação. Nesse caso M aceitaria uma cadeia somente se ambas M_1 *e* M_2 a aceitassem; portanto a linguagem resultante seria a *interseção* e não a união. Na verdade, esse resultado prova que a classe de linguagens regulares é fechada sob interseção.

TEOREMA 1.26

A classe de linguagens regulares é fechada sob a operação de concatenação.

Em outras palavras, se A_1 e A_2 são linguagens regulares, então o mesmo acontece com $A_1 \circ A_2$.

Para provar esse teorema, vamos tentar algo na linha da prova do caso da união. Tal qual anteriormente, podemos começar com os autômatos finitos M_1 e M_2 que reconhecem as linguagens regulares A_1 e A_2. Mas agora, em vez de construir o autômato M para aceitar sua entrada se M_1 ou M_2 aceitam, ele tem que aceitar se sua entrada puder ser quebrada em duas partes, sendo que M_1 aceita a primeira parte e M_2 aceita a segunda parte. O problema é que M não sabe onde quebrar sua entrada (isto é, onde a primeira parte termina e a segunda parte começa). Para resolver esse problema introduzimos uma nova técnica chamada não-determinismo.

1.2 NÃO-DETERMINISMO

Não-determinismo é um conceito útil que tem tido grande impacto sobre a teoria da computação. Até agora em nossa discussão, todo passo de uma computação segue de uma maneira única do passo precedente. Quando a máquina está em um dado estado e lê o próximo símbolo de entrada, sabemos qual será o próximo estado — está determinado. Chamamos isso de computação ***determinística***. Em uma máquina ***não-determinística***, várias escolhas podem existir para o próximo estado em qualquer ponto.

Não-determinismo é uma generalização de determinismo; portanto, todo autômato finito determinístico é automaticamente um autômato finito não-determinístico. Como a Figura 1.27 mostra, autômatos finitos não-determinísticos podem ter características adicionais.

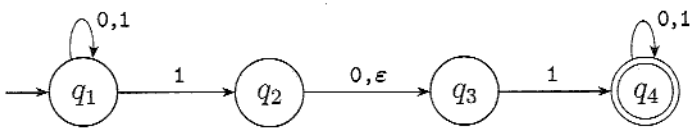


FIGURA 1.27
O autômato finito não-determinístico N_1.

A diferença entre um autômato finito determinístico, abreviado AFD, e um autômato finito não-determinístico, abreviado AFN, é imediatamente aparente. Primeiro, todo estado de um AFD sempre tem exatamente uma seta de transição saindo para cada símbolo no alfabeto. O autômato não-determinístico mostrado na Figura 1.27 viola essa regra. O estado q_1 tem uma seta saindo para 0, mas tem duas para 1; q_2 tem uma seta para 0, mas nenhuma para 1. Em um AFN um estado pode ter zero, uma ou várias setas saindo para cada símbolo do alfabeto.

Segundo, em um AFD, os rótulos sobre as setas de transição são símbolos do alfabeto. Esse AFN tem uma seta com o rótulo ε. Em geral, um AFN pode ter setas rotuladas com membros do alfabeto ou com ε. Zero, uma ou mais setas podem sair de cada estado com o rótulo ε.

Como um AFN computa? Suponha que você esteja rodando um AFN sobre uma cadeia de entrada e venha para um estado com múltiplas maneiras de prosseguir. Por exemplo, digamos que estamos no estado q_1 no AFN N_1 e que o próximo símbolo de entrada seja um 1. Após ler esse símbolo, a máquina divide-se em múltiplas cópias de si mesma e segue *todas* as possibilidades em paralelo. Cada cópia da máquina toma uma das maneiras possíveis de proceder e continua como antes. Se existirem escolhas subseqüentes, a máquina divide-se novamente. Se o próximo símbolo de entrada não aparece sobre qualquer das setas saindo do estado ocupado por uma cópia da máquina, essa cópia da máquina morre, juntamente com o ramo da computação associado a ela. Finalmente, se *qualquer uma* dessas cópias da máquina está em um estado de aceitação no final da entrada, o AFN aceita a cadeia de entrada.

Se um estado com um símbolo ε sobre uma seta saindo do mesmo for encontrado, algo semelhante acontece. Sem ler qualquer entrada, a máquina divide-se em múltiplas cópias, uma seguindo cada uma das setas saindo rotuladas com ε e uma permanecendo no estado corrente. Então a máquina prossegue não-deterministicamente como antes.

O não-determinismo pode ser visto como uma espécie de computação paralela na qual múltiplos e independentes "processos" ou "*threads*" podem estar rodando concorrentemente. Quando o AFN se divide para seguir as diversas escolhas, isso corresponde a um processo "bifurcar" em vários filhos, cada um procedendo separadamente. Se pelo menos um desses processos aceita, então a computação inteira aceita.

Outra maneira de pensar em uma computação não-determinística é como uma árvore de possibilidades. A raiz da árvore corresponde ao início da computação. Todo ponto de ramificação na árvore corresponde a um ponto na computação no qual a máquina tem múltiplas escolhas. A máquina aceita se pelo menos um dos ramos da computação termina em um estado de aceitação, como mostrado na Figura 1.28.

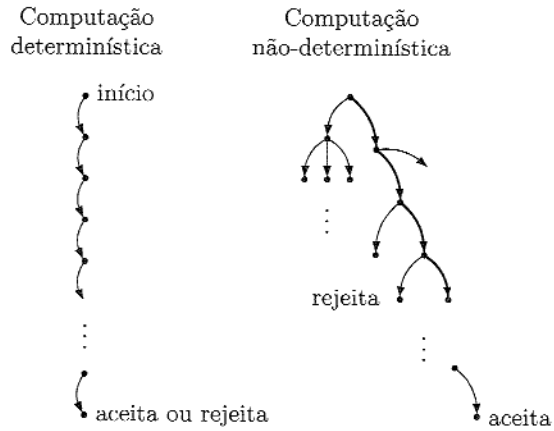


FIGURA 1.28
Computações determinísticas e não-determinísticas com um ramo de aceitação.

Vamos considerar algumas amostras de execuções do AFN N_1 mostrado na Figura 1.27. A computação de N_1 sobre a entrada 010110 é ilustrada na Figura 1.29.

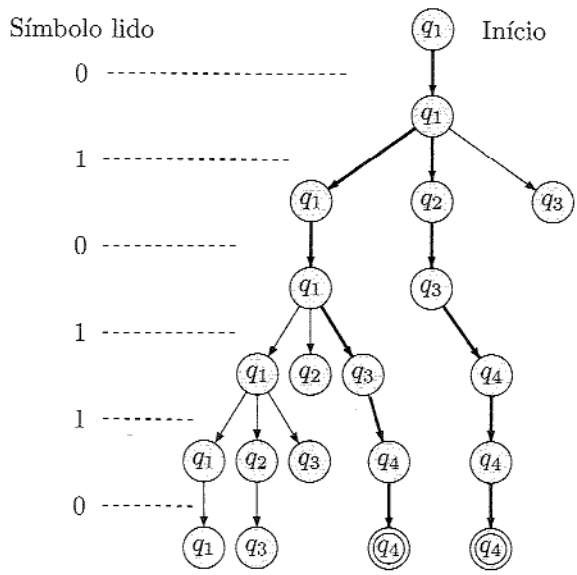


FIGURA 1.29
A computação de N_1 sobre a entrada 010110.

Para a entrada 010110, comece no estado inicial q_1 e leia o primeiro símbolo, 0. A partir de q_1 existe apenas um lugar para ir sobre um 0 — a saber, de volta a q_1; portanto, permaneça aí. Em seguida leia o segundo símbolo, 1. Em q_1 sobre um 1 há duas escolhas: ou permaneça em q_1 ou mova-se para q_2. Não-deterministicamente, a máquina se divide em duas para seguir cada uma das escolhas. Mantenha registro das possibilidades colocando um dedo sobre cada estado onde a máquina poderia estar. Portanto, você agora tem dedos sobre os estados q_1 e q_2. Uma seta ε sai do estado q_2 de modo que a máquina se divide novamente; mantenha um dedo sobre q_2, e mova o outro para q_3. Você agora tem dedos sobre q_1, q_2 e q_3.

Quando o terceiro símbolo, 0, for lido, considere cada um dos dedos separadamente. Mantenha o dedo sobre q_1 no lugar, mova o dedo sobre q_2 para q_3 e remova o dedo que tinha estado sobre q_3. Esse último dedo não tinha seta 0 para seguir e corresponde a um processo que simplesmente "morre." Nesse ponto você tem dedos sobre os estados q_1 e q_3.

Quando o quarto símbolo, 1, for lido, substitua o dedo sobre q_1 por dedos sobre os estados q_1 e q_2, e aí então substitua novamente o dedo sobre q_2 para seguir a seta ε para q_3, e mova o dedo que estava sobre q_3 para q_4. Você agora tem um dedo sobre cada um dos quatro estados.

Quando o quinto símbolo, 1, for lido, os dedos sobre q_1 e q_3 resultam em dedos sobre os estados q_1, q_2, q_3 e q_4, como você viu com o quarto símbolo. O dedo sobre o estado q_2 é removido. O dedo que estava sobre q_4 permanece sobre q_4. Agora você tem dois dedos sobre q_4; portanto, remova um, porque você só precisa lembrar que q_4 é um estado possível nesse ponto, não que ele é possível por múltiplas razões.

Quando o sexto e último símbolo, 0, é lido, mantenha o dedo sobre q_1 no lugar, mova o que está sobre q_2 para q_3, remova o que estava sobre q_3 e deixe o que está sobre q_4 no seu lugar. Você está agora no final da cadeia, e aceita se algum dedo estiver sobre um estado de aceitação. Você tem dedos sobre os estados q_1, q_3 e q_4, e como q_4 é um estado de aceitação, N_1 aceita essa cadeia.

O que é que N_1 faz sobre a entrada 010? Comece com um dedo sobre q_1. Depois de ler o 0, você ainda tem um dedo somente sobre q_1, mas depois do 1 existem dedos sobre q_1, q_2 e q_3 (não esqueça da seta ε). Depois do terceiro símbolo, 0, remova o dedo sobre q_3, mova o dedo sobre q_2 para q_3, e deixe o dedo sobre q_1 onde ele está. Nesse ponto, você está no final da entrada, e como nenhum dedo está sobre um estado de aceitação, N_1 rejeita essa entrada.

Continuando a experimentar dessa maneira, você verá que N_1 aceita todas as cadeias que contêm 101 ou 11 como uma subcadeia.

Os autômatos finitos não-determinísticos são úteis em vários sentidos. Como mostraremos, todo AFN pode ser convertido num AFD equivalente, e construir AFNs é às vezes mais fácil que construir diretamente AFDs. Um AFN pode ser muito menor que sua contrapartida determinística, ou seu funcionamento pode ser mais fácil de entender. Não-determinismo em autômatos finitos é também uma boa introdução a não-determinismo em modelos computacionais mais poderosos porque autômatos finitos são especialmente fáceis de entender. Agora nos voltamos para diversos exemplos de AFNs.

EXEMPLO 1.30

Seja A a linguagem consistindo de todas as cadeias sobre $\{0,1\}$ contendo um 1 na terceira posição a partir do final (por exemplo, 000100 está em A, mas 0011 não). O seguinte AFN de quatro estados, N_2, reconhece A.

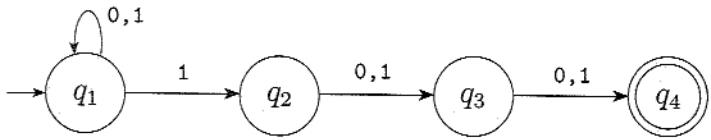


FIGURA 1.31
O AFN N_2 que reconhece A.

Uma boa maneira de ver a computação desse AFN é dizer que ele permanece no estado inicial q_1 até que "adivinhe" que está a três posições do final. Nesse ponto, se o símbolo de entrada for 1, ele ramifica para o estado q_2 e usa q_3 e q_4 para "checar" se sua adivinhação estava correta.

Conforme mencionado, todo AFN pode ser convertido num AFD equivalente, mas às vezes esse AFD pode ter muito mais estados. O menor AFD para A contém oito estados. Além disso, entender o funcionamento do AFN é muito mais fácil, como você pode ver examinando a Figura 1.32 para o AFD.

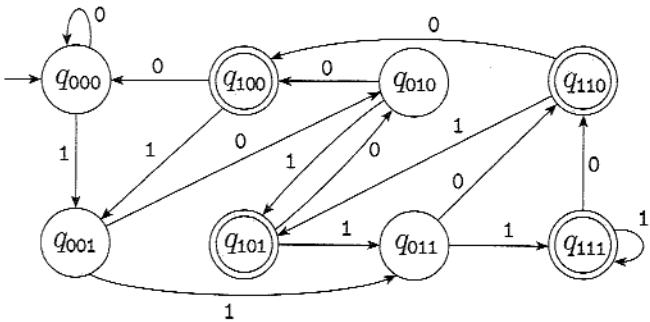


FIGURA 1.32
Um AFD que reconhece A.

Suponha que adicionássemos ε aos rótulos sobre as setas indo de q_2 para q_3 e de q_3 para q_4 na máquina N_2 da Figura 1.31. Assim, ambas as setas teriam

então o rótulo $0, 1, \varepsilon$ em vez de $0, 1$. Que linguagem N_2 reconheceria com essa modificação? Tente modificar o AFD da Figura 1.32 para reconhecer essa linguagem. ■

EXEMPLO 1.33

Considere o seguinte AFN N_3 que tem um alfabeto de entrada $\{0\}$ consistindo de um único símbolo. Um alfabeto contendo somente um símbolo é chamado ***alfabeto unário***.

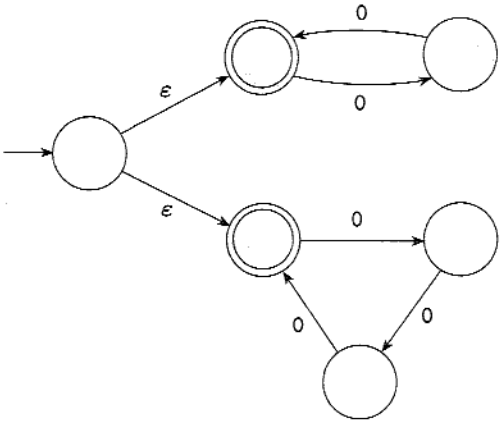


FIGURA 1.34
O AFN N_3.

Essa máquina demonstra a conveniência de se ter setas ε. Ela aceita todas as cadeias da forma 0^k, onde k é um múltiplo de 2 ou 3. (Lembre-se de que o expoente denota repetição, e não exponenciação numérica.) Por exemplo, N_3 aceita as cadeias ε, 00, 000, 0000 e 000000, mas não 0 ou 00000.

Pense na máquina inicialmente adivinhando se deve testar por um múltiplo de 2 ou um múltiplo de 3, ramificando ou no laço superior ou no laço inferior, e aí então verificando se sua adivinhação foi correta. É claro que poderíamos substituir essa máquina por uma que não tivesse setas ε ou mesmo nenhum não-determinismo, mas a máquina mostrada é a mais fácil de entender para essa linguagem. ■

EXEMPLO 1.35

Damos outro exemplo de um AFN na Figura 1.36. Pratique com ele para se convencer de que ele aceita as cadeias ε, a, baba e baa, mas não aceita as cadeias b, bb e babba. Adiante, usamos essa máquina para ilustrar o procedimento para converter AFNs para AFDs.

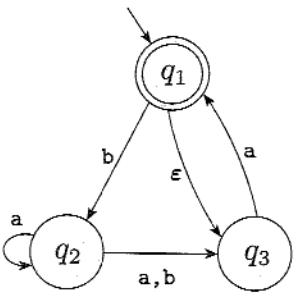


FIGURA 1.36
O AFN N_4.

DEFINIÇÃO FORMAL DE UM AUTÔMATO FINITO NÃO-DETERMINÍSTICO

A definição formal de um autômato finito não-determinístico é similar àquela de um autômato finito determinístico. Ambos têm estados, um alfabeto de entrada, uma função de transição, um estado inicial e uma coleção de estados de aceitação. Entretanto, eles diferem de uma maneira essencial: no tipo de função de transição. Em um AFD a função de transição toma um estado e um símbolo de entrada e produz o próximo estado. Em um AFN a função de transição toma um estado e um símbolo de entrada *ou a cadeia vazia* e produz *o conjunto de próximos estados possíveis*. Para escrever a definição formal, precisamos fixar alguma notação adicional. Para qualquer conjunto Q escrevemos $\mathcal{P}(Q)$ como sendo a coleção de todos os subconjuntos de Q. Aqui $\mathcal{P}(Q)$ é chamado ***conjunto das partes*** de Q. Para qualquer alfabeto Σ escrevemos Σ_ε como sendo $\Sigma \cup \{\varepsilon\}$. Agora podemos escrever a definição formal do tipo da função de transição em um AFN como $\delta\colon Q \times \Sigma_\varepsilon \longrightarrow \mathcal{P}(Q)$.

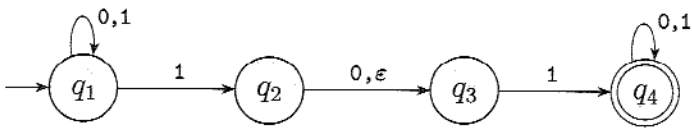
DEFINIÇÃO 1.37

Um ***autômato finito não-determinístico*** é uma 5-upla $(Q, \Sigma, \delta, q_0, F)$, onde

1. Q é um conjunto finito de estados,
2. Σ é um alfabeto finito,
3. $\delta\colon Q \times \Sigma_\varepsilon \longrightarrow \mathcal{P}(Q)$ é a função de transição,
4. $q_0 \in Q$ é o estado inicial, e
5. $F \subseteq Q$ é o conjunto de estados de aceitação.

EXEMPLO 1.38

Retomemos o AFN N_1:



A descrição formal de N_1 é $(Q, \Sigma, \delta, q_1, F)$, onde

1. $Q = \{q_1, q_2, q_3, q_4\}$,
2. $\Sigma = \{0,1\}$,
3. δ é dado como

	0	1	ε
q_1	$\{q_1\}$	$\{q_1, q_2\}$	$\emptyset$
q_2	$\{q_3\}$	$\emptyset$	$\{q_3\}$
q_3	$\emptyset$	$\{q_4\}$	$\emptyset$
q_4	$\{q_4\}$	$\{q_4\}$	$\emptyset$

,

4. q_1 é o estado inicial, e
5. $F = \{q_4\}$.

A definição formal de computação para um AFN é similar àquela para um AFD. Seja $N = (Q, \Sigma, \delta, q_0, F)$ um AFN e w uma cadeia sobre o alfabeto Σ. Então dizemos que N ***aceita*** w se podemos escrever w como $w = y_1 y_2 \cdots y_m$, onde cada y_i é um membro de Σ_ε e existe uma seqüência de estados $r_0, r_1, \ldots, r_m$ em Q com três condições:

1. $r_0 = q_0$,
2. $r_{i+1} \in \delta(r_i, y_{i+1})$, para $i = 0, \ldots, m-1$, e
3. $r_m \in F$.

A condição 1 diz que a máquina começa no estado inicial. A condição 2 diz que o estado r_{i+1} é um dos próximos estados permissíveis quando N está no estado r_i e lendo y_{i+1}. Observe que $\delta(r_i, y_{i+1})$ é o *conjunto* de próximos estados

permissíveis e portanto dizemos que r_{i+1} é um membro desse conjunto. Finalmente, a condição 3 diz que a máquina aceita sua entrada se o último estado é um estado de aceitação.

EQUIVALÊNCIA DE AFNS E AFDS

Os autômatos finitos determinísticos e não-determinísticos reconhecem a mesma classe de linguagens. Essa equivalência é, ao mesmo tempo, surpreendente e útil. É surpreendente porque AFNs parecem ter mais poder que AFDs e, portanto, poderíamos esperar que AFNs reconhecessem mais linguagens. É útil porque descrever um AFN para uma dada linguagem às vezes é muito mais fácil que descrever um AFD para essa linguagem.

Digamos que duas máquinas são ***equivalentes*** se elas reconhecem a mesma linguagem.

TEOREMA 1.39

Todo autômato finito não-determinístico tem um autômato finito determinístico equivalente.

IDÉIA DA PROVA Se uma linguagem é reconhecida por um AFN, então temos de mostrar a existência de um AFD que também a reconhece. A idéia é converter o AFN num AFD equivalente que simule o AFN.

Lembre-se da estratégia "leitor como autômato" para projetar autômatos finitos. Como você simularia o AFN se você estivesse fazendo de conta ser um AFD? O que você precisaria memorizar à medida que a cadeia de entrada é processada? Nos exemplos de AFNs você memorizou os vários ramos da computação colocando um dedo sobre cada estado que poderia estar ativo em dados pontos na entrada. Você atualizava a simulação movendo, adicionando e removendo dedos conforme a maneira pela qual o AFN opera. Tudo o que você precisava memorizar era o conjunto de estados colocando dedos sobre eles.

Se k é o número de estados do AFN, ele tem 2^k subconjuntos de estados. Cada subconjunto corresponde a uma das possibilidades de que o AFD tem de se lembrar; portanto, o AFD que simula o AFN terá 2^k estados. Agora precisamos descobrir qual será o estado inicial e os estados de aceitação do AFD, e qual será sua função de transição. Podemos discutir isso mais facilmente depois de fixar uma notação formal.

PROVA Seja $N = (Q, \Sigma, \delta, q_0, F)$ o AFN que reconhece alguma linguagem A. Construímos um AFD $M = (Q', \Sigma, \delta', q_0{}', F')$ que reconhece A. Antes de realizar a construção completa, vamos primeiro considerar o caso mais fácil no qual N não tem setas ε. Mais adiante levamos as setas ε em consideração.

1. $Q' = \mathcal{P}(Q)$.
   Todo estado de M é um conjunto de estados de N. Lembre-se de que $\mathcal{P}(Q)$ é o conjunto de subconjuntos de Q.

2. Para $R \in Q'$ e $a \in \Sigma$ seja $\delta'(R, a) = \{q \in Q | q \in \delta(r, a)$ para algum $r \in R\}$. Se R é um estado de M, é também um conjunto de estados de N. Quando M lê um símbolo a no estado R, ele mostra para onde a leva cada estado em R. Dado que cada estado pode ir para um conjunto de estados, tomamos a união de todos esses conjuntos. Outra maneira de escrever essa expressão é

$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a).$$ [4]

3. $q_0{}' = \{q_0\}$.
   M começa no estado correspondente à coleção contendo somente o estado inicial de N.
4. $F' = \{R \in Q' |\ R$ contém um estado de aceitação de $N\}$.
   A máquina M aceita se um dos possíveis estados nos quais N poderia estar nesse ponto é um estado de aceitação.

Agora precisamos considerar as setas ε. Para fazer isso fixamos um pouco mais de notação. Para qualquer estado R de M, definimos $E(R)$ como a coleção de estados que podem ser atingidos a partir de R indo somente ao longo de setas ε, incluindo os próprios membros de R. Formalmente, para $R \subseteq Q$ seja

$$E(R) = \{q |\ q \text{ pode ser atingido a partir de } R \text{ viajando-se ao longo de 0 ou mais setas } \varepsilon\}.$$

Então modificamos a função de transição de M para colocar dedos adicionais sobre todos os estados que podem ser atingidos indo ao longo de setas ε após cada passo. Substituindo $\delta(r, a)$ por $E(\delta(r, a))$ dá esse efeito. Conseqüentemente,

$$\delta'(R, a) = \{q \in Q |\ q \in E(\delta(r, a)) \text{ para algum } r \in R\}.$$

Adicionalmente precisamos de modificar o estado inicial de M para mover os dedos inicialmente para todos os estados possíveis que podem ser atingidos a partir do estado inicial de N ao longo das setas ε. Mudando $q_0{}'$ para $E(\{q_0\})$ dá esse efeito. Agora completamos a construção do AFD M que simula o AFN N.

A construção de M obviamente funciona corretamente. Em todo passo na computação de M sobre uma entrada, ela claramente entra num estado que corresponde ao subconjunto de estados nos quais N poderia estar nesse ponto. Assim, nossa prova está completa.

Se a construção usada na prova precedente fosse mais complexa precisaríamos provar que ela funciona como reivindicado. Geralmente essas provas procedem por indução sobre o número de passos da computação. A maioria das construções que usamos neste livro são imediatas e, portanto, não requerem tal prova de correção. Um exemplo de uma construção mais complexa que provamos ser de fato correta aparece na prova do Teorema 1.54.

---

[4] A notação $\bigcup_{r \in R} \delta(r, a)$ significa: a união dos conjuntos $\delta(r, a)$ para cada r em R.

O Teorema 1.39 afirma que todo AFN pode ser convertido em um AFD equivalente. Portanto, autômatos finitos não-determinísticos dão uma maneira alternativa de caracterizar as linguagens regulares. Enunciamos esse fato como um corolário do Teorema 1.39.

COROLÁRIO 1.40

Uma linguagem é regular se e somente se algum autômato finito não-determinístico a reconhece.

Uma direção da condição "se e somente se" afirma que uma linguagem é regular se algum AFN a reconhece. O Teorema 1.39 mostra que qualquer AFN pode ser convertido num AFD equivalente. Conseqüentemente, se um AFN reconhece uma dada linguagem, o mesmo acontece com algum AFD, e portanto a linguagem é regular. A outra direção da condição "se e somente se" afirma que uma linguagem é regular somente se algum AFN a reconhece. Ou seja, se uma linguagem é regular, algum AFN tem que reconhecê-la. Obviamente, essa condição é verdadeira porque uma linguagem regular tem um AFD reconhecendo-a e qualquer AFD é também um AFN.

EXEMPLO 1.41

Vamos ilustrar o procedimento que demos na prova do Teorema 1.39 para converter um AFN para um AFD usando a máquina N_4 que aparece no Exemplo 1.35. Em nome da clareza, renomeamos os estados de N_4 para $\{1, 2, 3\}$. Portanto, na descrição formal de $N_4 = (Q, \{\mathtt{a},\mathtt{b}\}, \delta, 1, \{1\})$, o conjunto de estados Q é $\{1, 2, 3\}$ como mostrado na Figura 1.42.

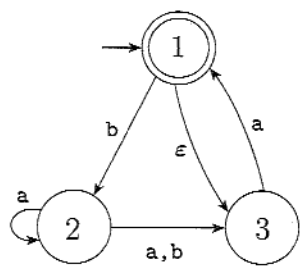


FIGURA 1.42
O AFN N_4.

Para construir um AFD D que seja equivalente a N_4, primeiro determinamos os estados de D. N_4 tem três estados, $\{1, 2, 3\}$; assim, construímos D com oito estados, um para cada subconjunto de estados de N_4. Rotulamos cada um dos estados de D com o subconjunto correspondente. Portanto, o conjunto de estados

de D é

$$\{\emptyset, \{1\}, \{2\}, \{3\}, \{1,2\}, \{1,3\}, \{2,3\}, \{1,2,3\}\}.$$

Em seguida, determinamos os estados inicial e de aceitação de D. O estado inicial é $E(\{1\})$, o conjunto de estados que são atingíveis a partir de 1 viajando ao longo de setas ε, mais o próprio 1. Uma seta ε vai de 1 para 3; portanto, $E(\{1\}) = \{1, 3\}$. Os novos estados de aceitação são aqueles contendo o estado de aceitação de N_4; assim, $\{\{1\}, \{1,2\}, \{1,3\}, \{1,2,3\}\}$.

Finalmente, determinamos a função de transição de D. Cada um dos estados de D vai para um lugar dada a entrada a e um lugar quando a entrada é b. Ilustramos o processo de se determinar a colocação das setas de transição de D com uns poucos exemplos.

Em D, o estado $\{2\}$ vai para $\{2,3\}$ na entrada a, porque em N_4, o estado 2 vai para ambos 2 e 3 na entrada a e não podemos ir mais longe a partir de 2 ou 3 ao longo de setas ε. O estado $\{2\}$ vai para o estado $\{3\}$ na entrada b, porque em N_4, o estado 2 vai apenas para o estado 3 na entrada b e não podemos ir mais longe a partir de 3 ao longo de setas ε.

O estado $\{1\}$ vai para $\emptyset$ na entrada a, porque nenhuma seta a sai dele. Ele vai para $\{2\}$ na entrada b. Note que o procedimento no Teorema 1.39 especifica que seguimos as setas ε *depois* que cada símbolo de entrada é lido. Um procedimento alternativo baseado em seguir as setas ε antes de ler cada entrada funciona igualmente bem, mas esse método não é ilustrado neste exemplo.

O estado $\{3\}$ vai para $\{1,3\}$ na entrada a, pois em N_4 o estado 3 vai para 1 na entrada a e 1, por sua vez, vai para 3 com uma seta ε. O estado $\{3\}$ na entrada b vai para $\emptyset$.

O estado $\{1,2\}$ na entrada a vai para $\{2,3\}$ porque 1 não aponta para nenhum estado com seta a e 2 aponta para ambos 2 e 3 com seta a e nenhum aponta para lugar algum com seta ε. O estado $\{1,2\}$ na entrada b vai para $\{2,3\}$. Continuando dessa maneira obtemos o seguinte diagrama para D.

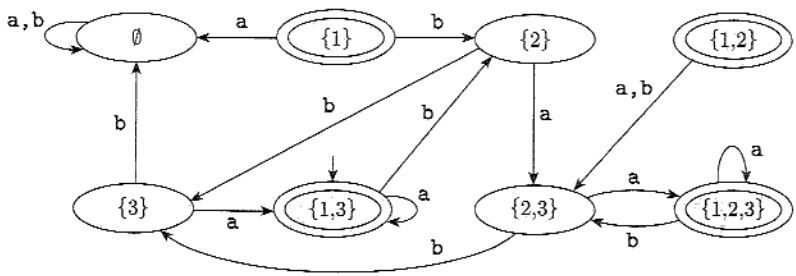


FIGURA 1.43
Um AFD D que é equivalente ao AFN N_4.

Podemos simplificar essa máquina observando que nenhuma seta aponta para os estados $\{1\}$ e $\{1, 2\}$, portanto eles podem ser removidos sem afetar o desem-

penho da máquina. Fazendo isso chegamos à Figura 1.44.

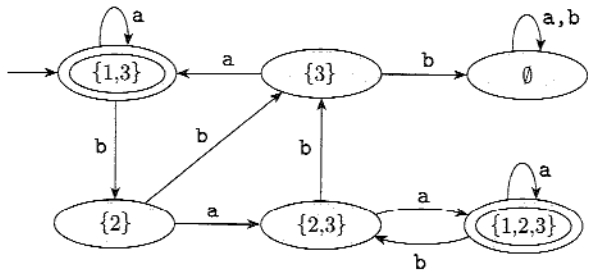


FIGURA 1.44
O AFD D após remover estados desnecessários.

FECHO SOB AS OPERAÇÕES REGULARES

Agora, retornamos ao fecho da classe de linguagens regulares sob as operações regulares que começamos na Seção 1.1. Nosso objetivo é provar que a união, a concatenação e a estrela de linguagens regulares são ainda regulares. Abandonamos a tentativa original de fazer isso quando vimos que lidar com a operação de concatenação era complicado demais. O uso de não-determinismo torna as provas muito mais fáceis.

Primeiro, vamos considerar novamente o fecho sob união. Antes, provamos o fecho sob união simulando deterministicamente ambas as máquinas simultaneamente via uma construção do produto cartesiano. Agora damos uma nova demonstração para ilustrar a técnica do não-determinismo. Revisar a primeira prova, que aparece na página 46, pode valer a pena para ver o quão mais fácil e mais intuitiva é a nova prova.

TEOREMA 1.45

A classe de linguagens regulares é fechada sob a operação de união.

IDÉIA DA PROVA Temos as linguagens regulares A_1 e A_2 e desejamos provar que $A_1 \cup A_2$ é regular. A idéia é tomar os dois AFNs, N_1 e N_2 para A_1 e A_2, e combiná-los em um novo AFN, N.

A máquina N tem de aceitar sua entrada se N_1 ou N_2 aceita essa entrada. A nova máquina tem um novo estado inicial que ramifica para os estados iniciais das máquinas anteriores com setas ε. Dessa maneira a nova máquina não-deterministicamente adivinha qual das duas máquinas aceita a entrada. Se uma delas aceita a entrada, N também a aceitará.

Representamos essa construção na Figura 1.46. À esquerda, indicamos o estado inicial e os estados de aceitação das máquinas N_1 e N_2 com cículos grandes e alguns estados adicionais com círculos pequenos. À direita, mostramos como combinar N_1 e N_2 adicionando setas de transição, resultando em N,.

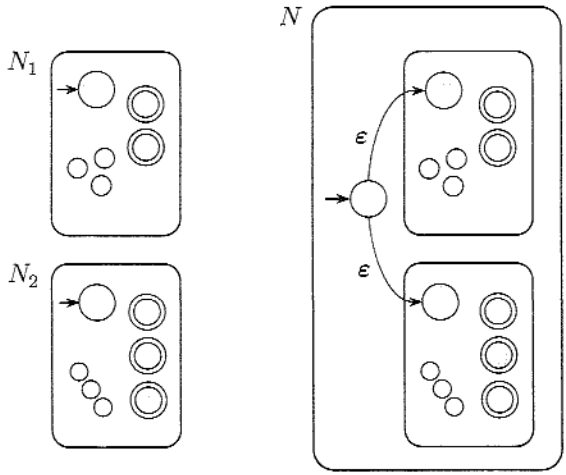


FIGURA 1.46
Construção de um AFN N para reconhecer $A_1 \cup A_2$.

PROVA

Suponha que $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ reconheça A_1, e que
$N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ reconheça A_2.

Construa $N = (Q, \Sigma, \delta, q_0, F)$ para reconhecer $A_1 \cup A_2$.

1. $Q = \{q_0\} \cup Q_1 \cup Q_2$.
   Os estados de N são todos os estados de N_1 e N_2, com a adição de um novo estado inicial q_0.
2. O estado q_0 é o estado inicial de N.
3. Os estados de aceitação $F = F_1 \cup F_2$.
   Os estados de aceitação de N são todos os estados de aceitação de N_1 e N_2. Dessa forma N aceita se N_1 aceita ou N_2 aceita.

4. Defina δ de modo que para qualquer $q \in Q$ e qualquer $a \in \Sigma_\varepsilon$,

$$\delta(q,a) = \begin{cases} \delta_1(q,a) & q \in Q_1 \\ \delta_2(q,a) & q \in Q_2 \\ \{q_1, q_2\} & q = q_0 \text{ e } a = \varepsilon \\ \emptyset & q = q_0 \text{ e } a \neq \varepsilon. \end{cases}$$

Agora podemos provar o fecho sob concatenação. Lembre-se de que anteriormente, sem não-determinismo, completar a prova teria sido difícil.

TEOREMA 1.47

A classe de linguagens regulares é fechada sob a operação de concatenação.

IDÉIA DA PROVA Temos as linguagens regulares A_1 e A_2 e desejamos provar que $A_1 \circ A_2$ é regular. A idéia é tomar dois AFNs, N_1 e N_2 para A_1 e A_2, e combiná-los em um novo AFN N como fizemos para o caso da união, mas dessa vez de uma maneira diferente, como mostrado na Figura 1.48.

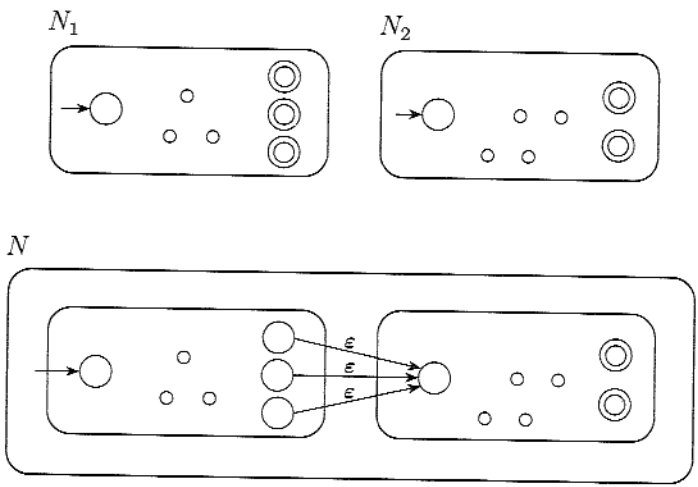


FIGURA 1.48
Construção de N para reconhecer $A_1 \circ A_2$.

Atribua ao estado inicial de N o estado inicial de N_1. Os estados de aceitação de N_1 têm setas ε adicionais que não-deterministicamente permitem ramificar para N_2 sempre que N_1 está num estado de aceitação, significando que ele encontrou uma parte inicial da entrada que constitui uma cadeia em A_1. Os estados de aceitação de N são somente os estados de aceitação de N_2. Por conseguinte, ele aceita quando a entrada pode ser dividida em duas partes, a primeira aceita por N_1 e a segunda por N_2. Podemos pensar em N como não-deterministicamente adivinhando onde fazer a divisão.

PROVA

Suponha que $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ reconheça A_1, e que
$N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ reconheça A_2.

Construa $N = (Q, \Sigma, \delta, q_1, F_2)$ para reconhecer $A_1 \circ A_2$.

1. $Q = Q_1 \cup Q_2$.
   Os estados de N são todos os estados de N_1 e N_2.
2. O estado q_1 é o mesmo que o estado inicial de N_1.
3. Os estados de aceitação F_2 são os mesmos que os estados de aceitação de N_2.
4. Defina δ de modo que para qualquer $q \in Q$ e qualquer $a \in \Sigma_\varepsilon$,

$$\delta(q,a) = \begin{cases} \delta_1(q,a) & q \in Q_1 \text{ e } q \notin F_1 \\ \delta_1(q,a) & q \in F_1 \text{ e } a \neq \varepsilon \\ \delta_1(q,a) \cup \{q_2\} & q \in F_1 \text{ e } a = \varepsilon \\ \delta_2(q,a) & q \in Q_2. \end{cases}$$

TEOREMA 1.49

A classe de linguagens regulares é fechada sob a operação estrela.

IDÉIA DA PROVA Temos uma linguagem regular A_1 e desejamos provar que A_1^* também é regular. Tomamos um AFN N_1 para A_1 e modificamo-lo para reconhecer A_1^*, como mostrado na Figura 1.50. O AFN resultante N aceitará sua entrada sempre que ela puder ser quebrada em várias partes e N_1 aceite cada uma das partes.

Podemos construir N como N_1 com setas ε adicionais retornando ao estado inicial a partir dos estados de aceitação. Dessa maneira, quando o processamento chega ao final de uma parte que N_1 aceita, a máquina N tem a opção de pular de volta para o estado inicial para tentar ler uma outra parte que N_1 aceite. Adicionalmente, temos que modificar N de tal forma que ele aceite ε, que é sempre um

membro de A_1^*. Uma idéia (levemente má) é simplesmente adicionar o estado inicial ao conjunto de estados de aceitação. Essa abordagem certamente adiciona ε à linguagem reconhecida, mas ela também pode adicionar outras cadeias indesejadas. O Exercício 1.15 pede um exemplo da falha dessa idéia. A maneira de consertar a construção é adicionar um novo estado inicial, que também seja um estado de aceitação, e que tenha uma seta ε para o antigo estado inicial. Essa solução tem o efeito desejado de adicionar ε à linguagem sem adicionar nada mais.

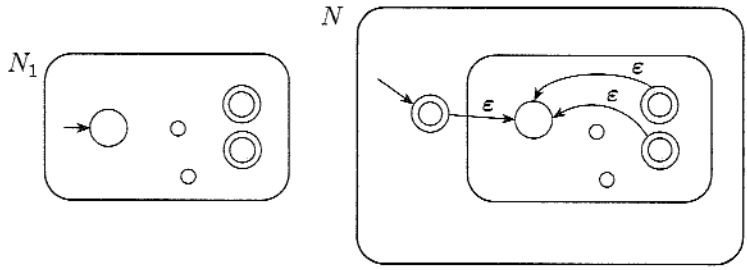


FIGURA 1.50
Construção de N para reconhecer A^*.

PROVA Suponha que $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ reconheça A_1. Construa $N = (Q, \Sigma, \delta, q_0, F)$ para reconhecer A_1^*.

1. $Q = \{q_0\} \cup Q_1$.
   Os estados de N são os estados de N_1 mais um novo estado inicial.
2. O estado q_0 é o novo estado inicial.
3. $F = \{q_0\} \cup F_1$.
   Os estados de aceitação são os antigos estados de aceitação mais o novo estado inicial.
4. Defina δ de modo que para qualquer $q \in Q$ e qualquer $a \in \Sigma_\varepsilon$,

$$\delta(q,a) = \begin{cases} \delta_1(q,a) & q \in Q_1 \text{ e } q \notin F_1 \\ \delta_1(q,a) & q \in F_1 \text{ e } a \neq \varepsilon \\ \delta_1(q,a) \cup \{q_1\} & q \in F_1 \text{ e } a = \varepsilon \\ \{q_1\} & q = q_0 \text{ e } a = \varepsilon \\ \emptyset & q = q_0 \text{ e } a \neq \varepsilon. \end{cases}$$

1.3 EXPRESSÕES REGULARES

Na aritmética, podemos usar as operações $+$ e $\times$ para montar expressões tais como

$$(5 + 3) \times 4 .$$

Similarmente, podemos usar as operações regulares para montar expressões descrevendo linguagens, que são chamadas ***expressões regulares***. Um exemplo é:

$$(0 \cup 1)0^* .$$

O valor da expressão aritmética é o número 32. O valor de uma expressão regular é uma linguagem. Nesse caso, o valor é a linguagem constituída de todas as cadeias começando com 0 ou 1 seguido por um número qualquer de 0s. Obtemos esse resultado dissecando a expressão em suas partes. Primeiro, os símbolos 0 e 1 são abreviações para os conjuntos $\{0\}$ e $\{1\}$. Dessa forma, $(0 \cup 1)$ significa $(\{0\} \cup \{1\})$. O valor dessa parte é a linguagem $\{0,1\}$. A parte 0^* significa $\{0\}^*$, e seu valor é a linguagem constituída de todas as cadeias contendo qualquer número de 0s. Segundo, como o símbolo $\times$ em álgebra, o símbolo da concatenação $\circ$ freqüentemente está ímplicito nas expressões regulares. Por conseguinte, $(0 \cup 1)0^*$ é, na realidade, uma abreviação de $(0 \cup 1) \circ 0^*$. A concatenação junta as cadeias das duas partes para obter o valor da expressão inteira.

As expressões regulares têm um papel importante em aplicações da ciência da computação. Em aplicações envolvendo texto, os usuários podem querer fazer busca por cadeias que satisfaçam certos padrões. Expressões regulares provêem um método poderoso para descrever tais padrões. Utilitários tais como AWK e GREP no UNIX, linguagens de programação modernas tais como PERL, e editores de texto, todos eles provêem mecanismos para a descrição de padrões usando expressões regulares.

EXEMPLO 1.51

Um outro exemplo de uma expressão regular é

$$(0 \cup 1)^*$$

Ela começa com a linguagem $(0 \cup 1)$ e aplica a operação $*$. O valor dessa expressão é a linguagem constituída de todas as possíveis cadeias de 0s e 1s. Se $\Sigma = \{0,1\}$, podemos escrever Σ como abreviação para a expressão regular $(0 \cup 1)$. Mais genericamente, se Σ for um alfabeto qualquer, a expressão regular Σ descreve a linguagem constituída de todas as cadeias de comprimento 1 sobre esse alfabeto, e Σ^* descreve a linguagem constituída de todas as cadeias sobre Σ. Similarmente, Σ^*1 é a linguagem que contém todas as cadeias que terminam em 1. A linguagem $(0\Sigma^*) \cup (\Sigma^*1)$ consiste de todas as cadeias que começam com 0 ou terminam com 1. ■

Na aritmética, dizemos que $\times$ tem precedência sobre $+$ querendo dizer que, quando existe uma escolha, fazemos a operação $\times$ primeiro. Assim, em $2 + 3 \times 4$ a operação 3×4 é feita antes da adição. Para fazer com que a adição seja feita primeiro temos que acrescentar parênteses para obter $(2+3) \times 4$. Em expressões regulares, a operação estrela é feita primeiro, seguida por concatenação e, finalmente, união, a menos que parênteses sejam usados para mudar a ordem usual.

DEFINIÇÃO FORMAL DE UMA EXPRESSÃO REGULAR

DEFINIÇÃO 1.52

Digamos que R é uma ***expressão regular*** se R for

1. a para algum a no alfabeto Σ,
2. ε,
3. $\emptyset$,
4. $(R_1 \cup R_2)$, onde R_1 e R_2 são expressões regulares,
5. $(R_1 \circ R_2)$, onde R_1 e R_2 são expressões regulares, ou
6. (R_1^*), onde R_1 é uma expressão regular.

Nos itens 1 e 2, as expressões regulares a e ε representam as linguagens $\{a\}$ e $\{\varepsilon\}$, respectivamente. No item 3, a expressão regular $\emptyset$ representa a linguagem vazia. Nos itens 4, 5 e 6, as expressões representam as linguagens obtidas tomando-se a união ou concatenação das linguagens R_1 e R_2, ou a estrela da linguagem R_1, respectivamente.

Não confunda as expressões regulares ε e $\emptyset$. A expressão ε representa a linguagem contendo uma única cadeia — a saber, a cadeia vazia — enquanto $\emptyset$ representa a linguagem que não contém nenhuma cadeia.

Aparentemente, estamos correndo o risco de definir a noção de expressão regular em termos de si própria. Se verdadeiro, teríamos uma ***definição circular***, o que seria inválido. Entretanto, R_1 e R_2 sempre são menores que R. Assim, estamos, na verdade, definindo expressões regulares em termos de expressões regulares menores e, dessa forma, evitando circularidade. Uma definição desse tipo é chamada ***definição indutiva***.

Os parênteses em uma expressão podem ser omitidos. Se isso acontecer, o cálculo é feito na ordem de precedência: estrela, e aí então concatenação, e depois união.

Por conveniência, tomamos R^+ como abreviação para RR^*. Em outras palavras, enquanto R^* tem todas as cadeias que são 0 ou mais concatenações de cadeias de R, a linguagem R^+ tem todas as cadeias que resultam de 1 ou mais

concatenações de cadeias de R. Portanto, $R^+ \cup \varepsilon = R^*$. Adicionalmente, tomamos R^k como abreviação para a concatenação de k Rs umas com as outras.

Quando queremos distinguir entre uma expressão regular R e a linguagem que ela descreve, escrevemos $L(R)$ como a linguagem de R.

EXEMPLO 1.53

Nas instâncias abaixo assumimos que o alfabeto Σ é $\{0,1\}$.

1. $0^*10^* = \{w \mid w$ contém um único $1\}$.
2. $\Sigma^*1\Sigma^* = \{w \mid w$ tem pelo menos um símbolo $1\}$.
3. $\Sigma^*001\Sigma^* = \{w \mid w$ contém a cadeia 001 como uma subcadeia$\}$.
4. $1^*(01^+)^* = \{w \mid$ todo 0 em w é seguido por pelo menos um $1\}$.
5. $(\Sigma\Sigma)^* = \{w \mid w$ é uma cadeia de comprimento par$\}$.[5]
6. $(\Sigma\Sigma\Sigma)^* = \{w \mid$ o comprimento de w é um múltiplo de três$\}$.
7. $01 \cup 10 = \{01, 10\}$.
8. $0\Sigma^*0 \cup 1\Sigma^*1 \cup 0 \cup 1 = \{w \mid w$ começa e termina com o mesmo símbolo$\}$.
9. $(0 \cup \varepsilon)1^* = 01^* \cup 1^*$.
   A expressão $0 \cup \varepsilon$ descreve a linguagem $\{0, \varepsilon\}$, portanto a operação de concatenação adiciona 0 ou ε antes de toda cadeia em 1^*.
10. $(0 \cup \varepsilon)(1 \cup \varepsilon) = \{\varepsilon, 0, 1, 01\}$.
11. $1^*\emptyset = \emptyset$.
    Concatenar o conjunto vazio a qualquer conjunto produz o conjunto vazio.
12. $\emptyset^* = \{\varepsilon\}$.
    A operação estrela junta qualquer número de cadeias da linguagem para obter uma cadeia no resultado. Se a linguagem for vazia, a operação estrela pode juntar 0 cadeias, dando apenas a cadeia vazia.

Se tomarmos R como uma expressão regular qualquer, temos as identidades a seguir. Elas são um bom teste para ver ser você entendeu a definição.

$R \cup \emptyset = R$.
Adicionar a linguagem vazia a qualquer outra linguagem não a modificará.

$R \circ \varepsilon = R$.
Juntar a cadeia vazia a qualquer outra cadeia não a modificará.

Entretanto, intercambiando $\emptyset$ e ε nas identidades precedentes pode fazer com que as igualdades falhem.

$R \cup \varepsilon$ pode não ser igual a R.
Por exemplo, se $R = 0$, então $L(R) = \{0\}$, mas $L(R \cup \varepsilon) = \{0, \varepsilon\}$.

[5] O ***comprimento*** de uma cadeia é o número de símbolos que ela contém.

$R \circ \emptyset$ pode não ser igual a R.
Por exemplo, se $R = 0$, então $L(R) = \{0\}$, mas $L(R \circ \emptyset) = \emptyset$.

As expressões regulares são ferramentas úteis no desenho de compiladores para linguagens de programação. Objetos elementares em uma linguagem de programação, chamados ***tokens***, tais como os nomes de variáveis e constantes, podem ser descritos com expressões regulares. Por exemplo, uma constante numérica que pode incluir uma parte fracionária e/ou um sinal pode ser descrita como um membro da linguagem

$$(+ \cup - \cup \varepsilon)\ (D^+ \cup D^+ . D^* \cup D^* . D^+)$$

onde $D = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ é o alfabeto de dígitos decimais. Exemplos de cadeias geradas são: 72, 3.14159, +7. e -.01 .

Uma vez que a sintaxe dos *tokens* da linguagem de programação tenha sido descrita com expressões regulares, sistemas automáticos podem gerar o ***analisador léxico***, a parte de um compilador que inicialmente processa o programa de entrada.

EQUIVALÊNCIA COM AUTÔMATOS FINITOS

Expressões regulares e autômatos finitos são equivalentes em seu poder descritivo. Esse fato é surpreendente porque autômatos finitos e expressões regulares aparentam superficialmente ser bastante diferentes. Entretanto, qualquer expressão regular pode ser convertida num autômato finito que reconhece a linguagem que ela descreve, e vice-versa. Lembre-se de que uma linguagem regular é uma que é reconhecida por algum autômato finito.

TEOREMA 1.54

Uma linguagem é regular se e somente se alguma expressão regular a descreve.

Este teorema tem duas direções. Enunciamos e provamos cada uma das direções como um lema separado.

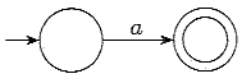
LEMA 1.55

Se uma linguagem é descrita por uma expressão regular, então ela é regular.

IDÉIA DA PROVA Vamos supor que tenhamos uma expressão regular R descrevendo alguma linguagem A. Mostramos como converter R num AFN que reconhece A. Pelo Corolário 1.40, se um AFN reconhece A, então A é regular.

PROVA Vamos converter R num AFN N. Consideramos os seis casos na descrição formal de expressões regulares.

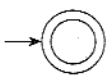
1. $R = a$ para algum a em Σ. Então $L(R) = \{a\}$, e o seguinte AFN reconhece $L(R)$.



   Note que essa máquina se encaixa na definição de um AFN mas não na de um AFD porque ela tem alguns estados sem nenhuma seta saindo para cada símbolo de entrada possível. É claro que poderíamos ter apresentado um AFD equivalente aqui, mas um AFN é tudo de que precisamos no momento, e ele é mais fácil de descrever.

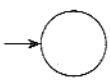
   Formalmente, $N = (\{q_1, q_2\}, \Sigma, \delta, q_1, \{q_2\})$, onde descrevemos δ dizendo que $\delta(q_1, a) = \{q_2\}$ e que $\delta(r, b) = \emptyset$ para $r \neq q_1$ ou $b \neq a$.

2. $R = \varepsilon$. Então $L(R) = \{\varepsilon\}$, e o seguinte AFN reconhece $L(R)$.



   Formalmente, $N = (\{q_1\}, \Sigma, \delta, q_1, \{q_1\})$, onde $\delta(r, b) = \emptyset$ para quaisquer r e b.

3. $R = \emptyset$. Então $L(R) = \emptyset$, e o seguinte AFN reconhece $L(R)$.



   Formalmente, $N = (\{q\}, \Sigma, \delta, q, \emptyset)$, onde $\delta(r, b) = \emptyset$ para quaisquer r e b.

4. $R = R_1 \cup R_2$.
5. $R = R_1 \circ R_2$.
6. $R = R_1^*$.

Para os três últimos casos, usamos as construções dadas nas provas de que a classe de linguagens regulares é fechada sob as operações regulares. Em outras palavras, construímos o AFN para R a partir dos AFNs para R_1 e R_2 (ou somente R_1 no caso 6) e a construção de fecho apropriada.

Isso conclui a primeira parte da prova do Teorema 1.54, dando a direção mais fácil da condição de se e somente se. Antes de seguir adiante para a outra

direção, vamos considerar alguns exemplos nos quais usamos esse procedimento para converter uma expressão regular em um AFN.

EXEMPLO 1.56

Convertemos a expressão regular (ab ∪ a)* em um AFN numa seqüência de estágios. Construimos a partir das subexpressões menores em direção às maiores até que tenhamos um AFN para a expressão original, como mostrado no diagrama da Figura 1.57. Note que esse procedimento geralmente não produz o AFN com o menor número de estados. Neste exemplo, o procedimento produz um AFN com oito estados, mas o menor AFN equivalente tem somente dois estados. Você pode encontrá-lo?

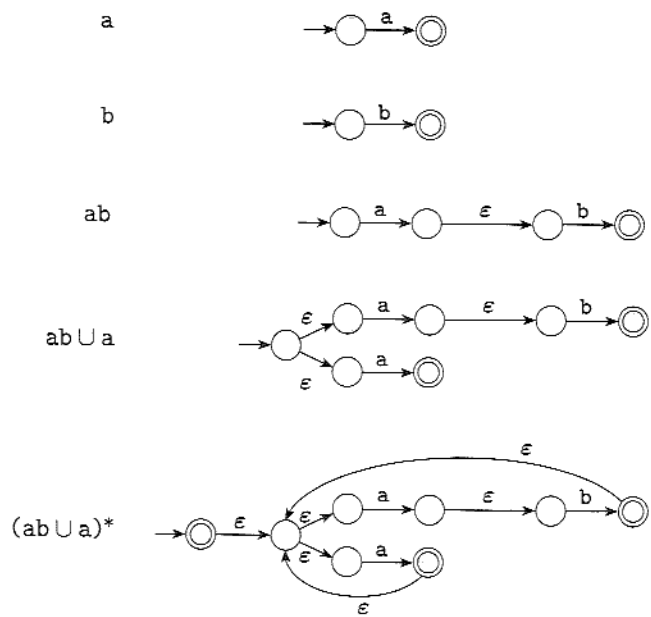


FIGURA 1.57
Construindo um AFN a partir da expressão regular (ab ∪ a)*.

EXEMPLO 1.58

Na Figura 1.59, convertemos a expressão regular (a∪b)*aba em um AFN. Alguns dos passos intermdediários não são mostrados.

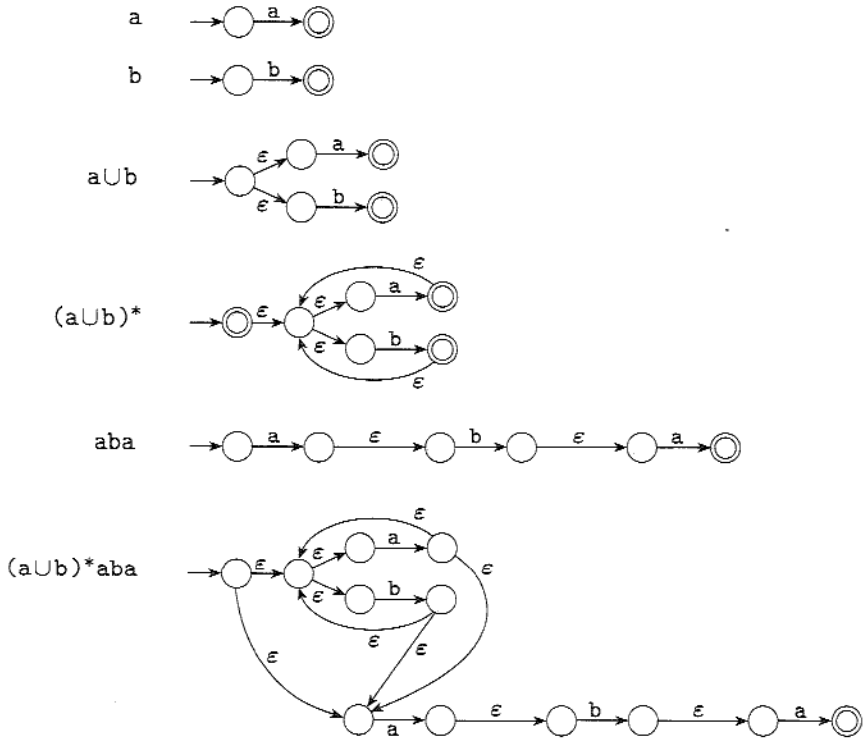


FIGURA 1.59
Construindo um AFN a partir da expressão (a ∪ b)*aba.

Agora vamos nos voltar para a outra direção da prova do Teorema 1.54.

LEMA 1.60

Se uma linguagem é regular, então ela é descrita por uma expressão regular.

IDÉIA DA PROVA Precisamos mostrar que, se uma linguagem A é regular, uma expressão regular a descreve. Pelo fato de A ser regular, ela é aceita por um AFD. Descrevemos um procedimento para converter AFDs em expressões regulares equivalentes.

Dividimos esse procedimento em duas partes, usando um novo tipo de autômato finito chamado ***automato finito não-determinístico generalizado***, AFNG. Primeiro, mostramos como converter AFDs em AFNGs, e depois AFNGs em expressões regulares.

Autômatos finitos não-determinísticos generalizados são simplesmente autômatos finitos não-determinísticos nos quais as setas de transição podem ter quaisquer expressões regulares como rótulos, em vez de apenas membros do alfabeto ou ε. O AFNG lê blocos de símbolos da entrada, não necessariamente apenas um símbolo de cada vez como em um AFN ordinário. O AFNG se move ao longo de uma seta de transição conectando dois estados ao ler um bloco de símbolos da entrada, o qual constitui uma cadeia descrita pela expressão regular sobre aquela seta. Um AFNG é não-determinístico e, portanto, pode ter várias maneiras diferentes de processar a mesma cadeia de entrada. Ele aceita a sua entrada se seu processamento puder levar o AFNG a estar em um estado de aceitação ao final da entrada. A figura a seguir apresenta um exemplo de um AFNG.

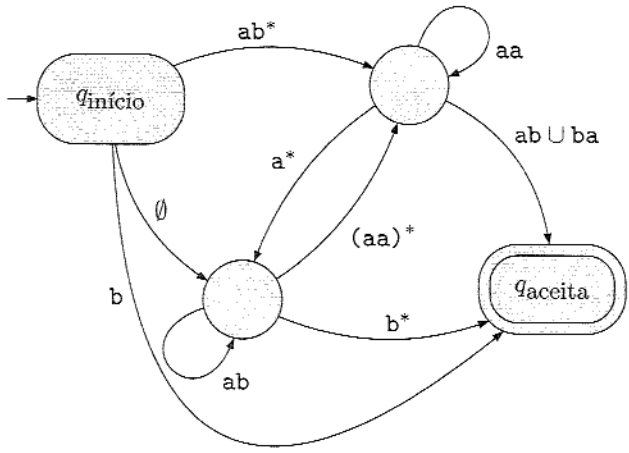


FIGURA 1.61
Um autômato finito não-determinístico generalizado.

Por conveniência, requeremos que os AFNGs tenham sempre um formato especial que atenda às seguintes condições:

- O estado inicial tem setas de transição saindo para todos os outros estados, mas nenhuma seta chegando de qualquer outro estado.
- Existe apenas um estado de aceitação, e ele tem setas chegando de todos os outros estados, mas nenhuma seta saindo para qualquer outro estado. Além disso, o estado de aceitação não é o mesmo que o estado inicial.
- Com exceção dos estados inicial e de aceitação, uma seta sai de cada estado para todos os outros e também de cada estado para ele mesmo.

Podemos converter facilmente um AFD em um AFNG no formato especial. Simplesmente, adicionamos um novo estado inicial com uma seta ε apontando para o estado inicial antigo e um novo estado de aceitação com setas ε chegando

dos estados de aceitação antigos. Se quaisquer setas têm múltiplos rótulos (ou se há múltiplas setas entre dois estados na mesma direção), substituímos cada uma por uma única seta cujo rótulo é a união dos rótulos anteriores. Finalmente, adicionamos setas com rótulos $\emptyset$ entre estados que não tenham setas. Este último passo não irá modificar a linguagem reconhecida porque uma transição rotulada com $\emptyset$ nunca pode ser utilizada. Daqui para frente, supomos que todos os AFNGs estarão no formato especial.

Agora mostramos como converter um AFNG em uma expressão regular. Digamos que o AFNG tenha k estados. Então, como um AFNG deve ter um estado inicial e um estado de aceitação e eles devem ser diferentes um do outro, sabemos que $k \geq 2$. Se $k > 2$, construímos um AFNG equivalente com $k - 1$ estados. Este passo pode ser repetido sobre o novo AFNG até que ele esteja reduzido a dois estados. Se $k = 2$, o AFNG tem uma única seta, que vai do estado inicial para o estado de aceitação. O rótulo dessa seta é a expressão regular equivalente. Por exemplo, os estágios na conversão de um AFD com três estados em uma expressão regular equivalente estão mostrados na figura a seguir.

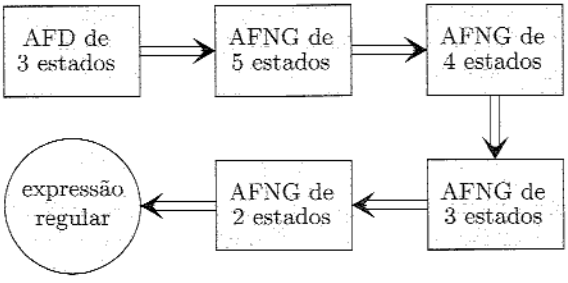


FIGURA 1.62
Estágios típicos na conversão de um AFD em uma expressão regular.

O passo crucial está na construção de um AFNG equivalente com um estado a menos quando $k > 2$. Fazemos isso selecionando um estado, removendo-o da máquina e reparando o resto de forma que seja reconhecida ainda a mesma linguagem. Qualquer estado serve, desde que não seja o estado inicial ou o estado de aceitação. Temos a garantia de que tal estado existe, visto que $k > 2$. Chamemos o estado removido de q_{rem}.

Após remover q_{rem} reparamos a máquina alterando as expressões regulares que rotulam cada uma das setas restantes. Os novos rótulos compensam a ausência de q_{rem} adicionando de volta as computações perdidas. O novo rótulo indo de um estado q_i para um estado q_j é uma expressão regular que descreve todas as cadeias que levariam a máquina de q_i para q_j, seja diretamente, seja passando por q_{rem}. Ilustramos essa abordagem na Figura 1.63.

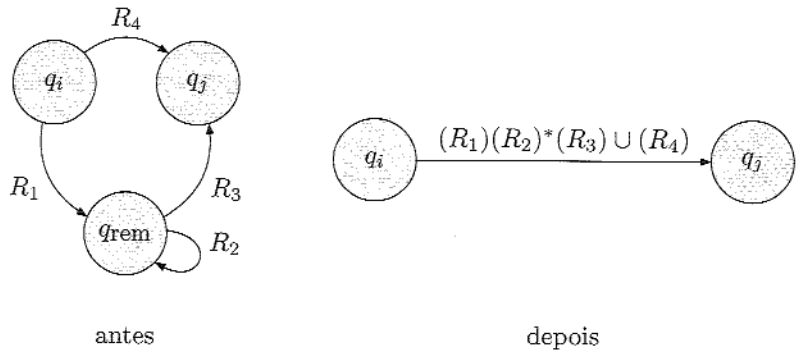


FIGURA 1.63
Construindo um AFNG equivalente com um estado a menos.

Na máquina antiga se q_i vai para q_{rem} com uma seta de rótulo R_1, q_{rem} vai para si mesmo com uma seta de rótulo R_2, q_{rem} vai para q_j com uma seta de rótulo R_3 e q_i vai para q_j com uma seta de rótulo R_4, então na nova máquina a seta de q_i para q_j recebe o rótulo

$$(R_1)(R_2)^*(R_3) \cup (R_4).$$

Fazemos essa mudança para toda seta indo de cada estado q_i para cada estado q_j, incluindo o caso em que $q_i = q_j$. A nova máquina reconhece a linguagem original.

PROVA Vamos agora realizar essa idéia formalmente. Primeiro, para facilitar a prova, definimos formalmente o novo tipo de autômato introduzido. Um AFNG é semelhante a um autômato finito não-determinístico, exceto pela função de transição, que tem a forma

$$\delta\colon \big(Q - \{q_{\text{aceita}}\}\big) \times \big(Q - \{q_{\text{início}}\}\big) \longrightarrow \mathcal{R}.$$

O símbolo $\mathcal{R}$ é a coleção de todas as expressões regulares sobre o alfabeto Σ, e $q_{\text{início}}$ e q_{aceita} são os estados inicial e de aceitação. Se $\delta(q_i, q_j) = R$, a seta do estado q_i para o estado q_j tem a expressão regular R como seu rótulo. O domínio da função de transição é $\big(Q - \{q_{\text{aceita}}\}\big) \times \big(Q - \{q_{\text{início}}\}\big)$, porque uma seta conecta todo estado a todos os outros estados, exceto que nenhuma seta sai de q_{aceita} ou chega em $q_{\text{início}}$.

DEFINIÇÃO 1.64

Um ***autômato finito não-determinístico generalizado*** é uma 5-upla,
$(Q, \Sigma, \delta, q_{\text{início}}, q_{\text{aceita}})$, onde

1. Q é o conjunto finito de estados,
2. Σ é o alfabeto de entrada,
3. $\delta\colon (Q - \{q_{\text{aceita}}\}) \times (Q - \{q_{\text{início}}\}) \longrightarrow \mathcal{R}$ é a função de transição,
4. $q_{\text{início}}$ é o estado inicial, e
5. q_{aceita} é o estado de aceitação.

Um AFNG aceita uma cadeia w em Σ^* se $w = w_1 w_2 \cdots w_k$, onde cada w_i está em Σ^*, e existe uma seqüência de estados $q_0, q_1, \ldots, q_k$ tal que

1. $q_0 = q_{\text{início}}$ é o estado inicial,
2. $q_k = q_{\text{aceita}}$ é o estado de aceitação, e
3. para cada i, temos $w_i \in L(R_i)$, onde $R_i = \delta(q_{i-1}, q_i)$; em outras palavras, R_i é a expressão sobre a seta de q_{i-1} a q_i.

Voltando à prova do Lema 1.60, seja M o AFD para a linguagem A. Então convertemos M em um AFNG G adicionando um novo estado inicial e um novo estado de aceitação e setas de transição adicionais conforme necessário. Usamos o procedimento CONVERT(G), que toma um AFNG como entrada e retorna uma expressão regular equivalente. Esse procedimento usa ***recursão***, o que significa que ele chama a si mesmo. Um laço infinito é evitado, visto que o procedimento chama a si mesmo somente para processar um AFNG que tem um estado a menos. O caso em que o AFNG tem dois estados é tratado sem recursão.

CONVERT(G):

1. Seja k o número de estados de G.
2. Se $k = 2$, então G deve consistir de um estado inicial, um estado de aceitação, e uma única seta conectando os dois rotulada com uma expressão regular R.
   Retorne a expressão R.
3. Se $k > 2$, selecionamos qualquer $q_{\text{rem}} \in Q$ diferente de $q_{\text{início}}$ e de q_{aceita} e seja G' o AFNG $(Q', \Sigma, \delta', q_{\text{início}}, q_{\text{aceita}})$, onde
   $$Q' = Q - \{q_{\text{rem}}\},$$
   e para qualquer $q_i \in Q' - \{q_{\text{aceita}}\}$ e qualquer $q_j \in Q' - \{q_{\text{início}}\}$ seja
   $$\delta'(q_i, q_j) = (R_1)(R_2)^*(R_3) \cup (R_4),$$
   para $R_1 = \delta(q_i, q_{\text{rem}})$, $R_2 = \delta(q_{\text{rem}}, q_{\text{rem}})$, $R_3 = \delta(q_{\text{rem}}, q_j)$ e $R_4 = \delta(q_i, q_j)$.

4. Compute CONVERT(G') e retorne esse valor.

A seguir, provamos que CONVERT retorna um valor correto.

AFIRMATIVA 1.65

Para qualquer AFNG G, CONVERT(G) é equivalente a G.

Provamos essa afirmação por indução sobre k, o número de estados do AFNG.

Base: Prove que a afirmação é verdadeira para $k = 2$ estados. Se G tem apenas dois estados, ele só pode ter uma única seta, que vai do estado inicial para o estado de aceitação. A expressão regular que é o rótulo sobre essa seta descreve todas as cadeias que propiciam a G chegar ao estado de aceitação. Logo, essa expressão é equivalente a G.

Passo da Indução: Suponha que a afirmação seja verdadeira para $k - 1$ estados e use essa suposição para provar que a afirmação é verdadeira para k estados. Primeiro, mostramos que G e G' reconhecem a mesma linguagem. Suponha que G aceite uma entrada w. Então, em um ramo de aceitação da computação G entra em uma seqüência de estados:

$$q_{\text{início}}, q_1, q_2, q_3, \ldots, q_{\text{aceita}}.$$

Se nenhum deles é o estado removido q_{rem}, claramente G' também aceita w. A razão é que cada uma das novas expressões regulares rotulando as setas de G' contém a expressão regular antiga como parte de uma união.

Se q_{rem} aparece, removendo cada série de estados q_{rem} consecutivos, forma uma computação de aceitação para G'. Os estados q_i e q_j imediatamente antes e após uma série têm uma nova expressão regular na seta que os liga que descreve todas as cadeias que levam de q_i a q_j via q_{rem} sobre G. Portanto, G' aceita w.

Para a recíproca, suponha que G' aceite uma entrada w. Como cada seta entre quaisquer dois estados q_i e q_j em G' descreve a coleção das cadeias que levam de q_i a q_j em G, seja diretamente ou via q_{rem}, G tem de aceitar w também. Portanto, G e G' são equivalentes.

A hipótese de indução afirma que quando o algoritmo chama a si mesmo recursivamente sobre a entrada G', o resultado é uma expressão regular equivalente a G', porque G' tem $k - 1$ estados. Logo, essa expressão regular também é equivalente a G, e o algoritmo está provado correto.

Isso conclui a prova da Afirmativa 1.65, Lema 1.60 e Teorema 1.54.

EXEMPLO 1.66

Neste exemplo, usamos o algoritmo precedente para converter um AFD em uma expressão regular. Começamos com o AFD de dois estados da Figura 1.67(a).

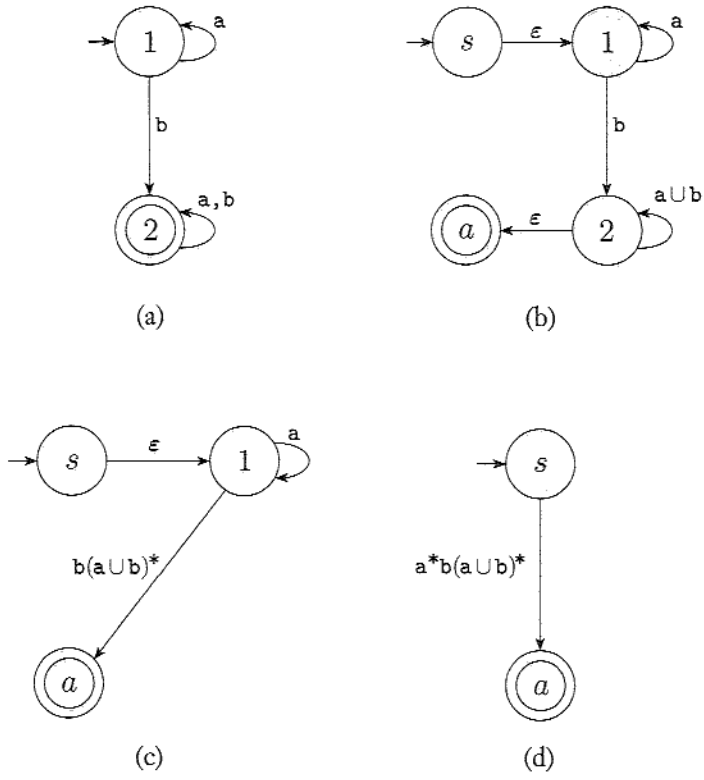


FIGURA 1.67
Convertendo um AFD de dois estados em uma expressão regular equivalente.

Na Figura 1.67(b), montamos um AFNG de quatro estados adicionando um novo estado inicial e um novo estado de aceitação, chamados s e a em vez de $q_{\text{início}}$ e q_{aceita}, de modo que possamos desenhá-los convenientemente. Para evitar carregar demais a figura, não traçamos as setas rotuladas $\emptyset$, muito embora elas estejam presentes. Note que substituímos o rótulo a, b sobre o laço no estado 2 do AFD pelo rótulo a ∪ b no ponto correspondente do AFNG. Fazemos isso porque o rótulo do AFD representa duas transições, uma para a e outra para b, enquanto o AFNG pode ter apenas uma transição indo de 2 para si mesmo.

Na Figura 1.67(c) removemos o estado 2 e atualizamos os rótulos das setas remanescentes. Neste caso, o único rótulo que muda é o de 1 para a. Na parte (b) o rótulo era $\emptyset$, mas na parte (c) é b(a ∪ b)*. Obtemos esse resultado seguindo o passo 3 do procedimento CONVERT. O estado q_i é o estado 1, o estado q_j é a, e q_{rem} é 2; assim, $R_1 = \mathtt{b}$, $R_2 = \mathtt{a} \cup \mathtt{b}$, $R_3 = \varepsilon$ e $R_4 = \emptyset$. Portanto, o novo rótulo sobre a seta de 1 para a é $(\mathtt{b})(\mathtt{a} \cup \mathtt{b})^*(\varepsilon) \cup \emptyset$. Simplificamos essa expressão regular para b(a ∪ b)*.

Na Figura 1.67(d) removemos o estado 1 da parte (c) e seguimos o mesmo procedimento. Como restam apenas os estados inicial e de aceitação, o rótulo sobre a seta que os une é a expressão regular equivalente ao AFD original.

EXEMPLO 1.68

Neste exemplo começamos com um AFD de três estados. Os passos da conversão são mostrados na Figura 1.69.

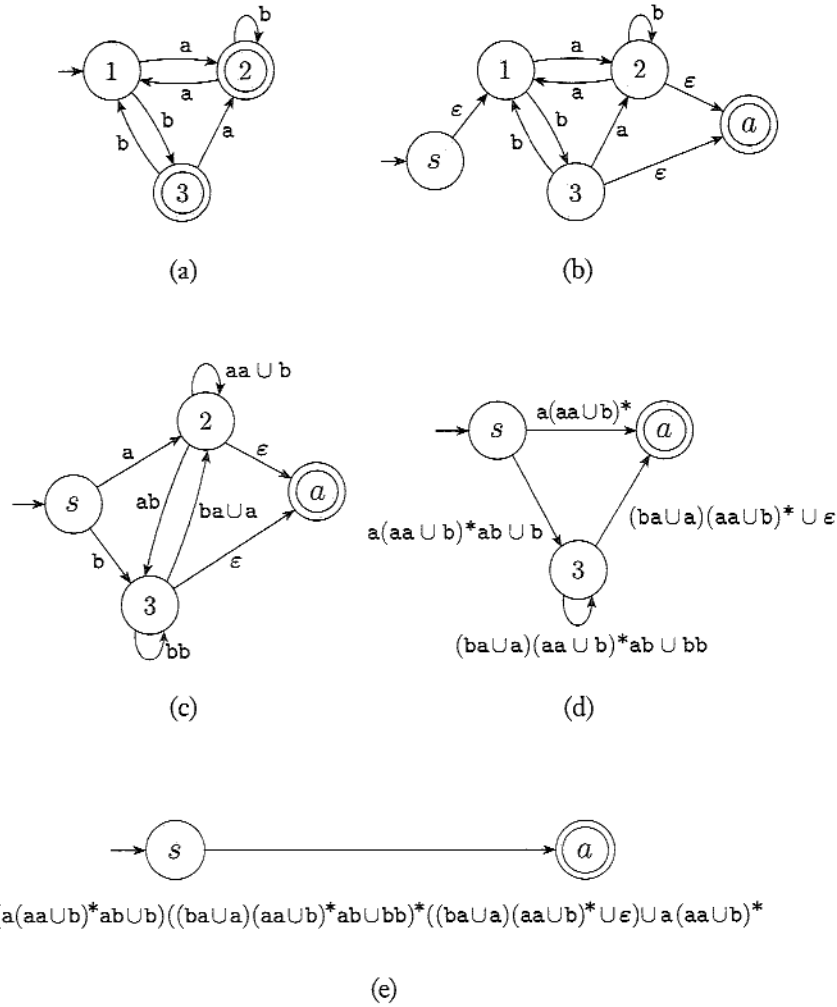


FIGURA 1.69
Convertendo um AFD de três estados em uma expressão regular equivalente.

1.4 LINGUAGENS NÃO-REGULARES

Para entender o poder dos autômatos finitos você precisa entender também suas limitações. Nesta seção, mostramos como provar que certas linguagens não podem ser reconhecidas por nenhum autômato finito.

Tomemos a linguagem $B = \{0^n 1^n | \ n \geq 0\}$. Se tentarmos encontrar um AFD que reconheça B, descobrimos que a máquina parece necessitar de lembrar quantos 0s foram vistos até então à medida que ela lê a entrada. Como o número de 0s não é limitado, a máquina terá de registrar um número ilimitado de possibilidades. Mas ela não pode fazer isso com qualquer quantidade finita de estados.

A seguir, apresentamos um método para provar que linguagens como B não são regulares. O argumento que acaba de ser apresentado já não prova não-regularidade baseando-se no fato de que o número de 0s é ilimitado? Não, não prova. Só porque a linguagem parece requerer memória ilimitada não significa que assim seja necessariamente. Isso acontece de ser verdadeiro para a linguagem B, mas outras linguagens parecem requerer um número ilimitado de possibilidades e, ainda assim, são regulares. Por exemplo, considere duas linguagens sobre o alfabeto $\Sigma = \{0,1\}$:

$C = \{w | \ w$ tem um número igual de 0s e 1s$\}$, e

$D = \{w | \ w$ tem um número igual de ocorrências de 01 e 10 como subcadeias$\}$.

À primeira vista, uma máquina reconhecedora parece ter que contar em ambos os casos, e portanto nenhuma das duas linguagens parece ser regular. Como esperado, C não é regular, mas surpreendentemente D é regular![6] Assim, nossa intuição pode às vezes nos desencaminhar, motivo pelo qual precisamos de provas matemáticas para ter certeza. Nesta seção, mostramos como provar que certas linguagens não são regulares.

O LEMA DO BOMBEAMENTO PARA LINGUAGENS REGULARES

Nossa técnica para provar não-regularidade provém de um teorema sobre linguagens regulares, tradicionalmente chamado ***lema do bombeamento***. Esse teorema afirma que todas as linguagens regulares têm uma propriedade especial. Se pudermos mostrar que uma linguagem não tem essa propriedade, temos a garantia de que ela não é regular. A propriedade enuncia que todas as cadeias da linguagem podem ser "bombeadas" se elas são no mínimo tão longas como

[6] Veja o Problema 1.48.

um determinado valor especial, denominado o ***comprimento de bombeamento***. Isto significa que cada uma dessas cadeias contém uma parte que pode ser repetida um número qualquer de vezes, com a cadeia resultante permanecendo na linguagem.

TEOREMA 1.70

Lema do bombeamento Se A é uma linguagem regular, então existe um número p (o comprimento de bombeamento) tal que, se s é qualquer cadeia de A de comprimento no mínimo p, então s pode ser dividida em três partes, $s = xyz$, satisfazendo as seguintes condições:

1. para cada $i \geq 0$, $xy^iz \in A$,
2. $|y| > 0$, e
3. $|xy| \leq p$.

Lembre-se da notação em que $|s|$ representa o comprimento da cadeia s, y^i significa que i cópias de y são concatenadas entre si e y^0 é igual a ε.

Quando s é dividida em xyz, ou x ou z pode ser ε, mas a condição 2 diz que $y \neq \varepsilon$. Observe que sem a condição 2 o teorema seria trivialmente verdadeiro. A condição 3 enuncia que as partes x e y juntas têm comprimento no máximo p. Ela é uma condição técnica extra que ocasionalmente achamos útil ao provar que certas linguagens não são regulares. Veja o Exemplo 1.74 para uma aplicação da condição 3.

IDÉIA DA PROVA Seja $M = (Q, \Sigma, \delta, q_1, F)$ um AFD que reconhece A. Atribuímos ao comprimento de bombeamento p o número de estados de M. Mostramos que qualquer cadeia s em A de comprimento pelo menos p pode ser quebrada nas três partes xyz satisfazendo as nossas três condições. E se nenhuma cadeia em A tem comprimento no mínimo p? Então nossa tarefa é ainda mais fácil, visto que o teorema se torna verdadeiro por *vacuidade*: obviamente, as três condições se verificam para todas as cadeias de comprimento no mínimo p se não existem tais cadeias.

Se s em A tem comprimento pelo menos p, considere a seqüência de estados pelos quais M passa quando computa com a entrada s. Ele começa no estado inicial, q_1, vai para, digamos, q_3, depois para, digamos, q_{20}, em seguida para q_9, e assim por diante, até que atinge o final de s no estado q_{13}. Com s em A, sabemos que M aceita s, portanto q_{13} é um estado de aceitação.

Se dissermos que n é o comprimento de s, a seqüência de estados $q_1, q_3, q_{20}, q_9, \ldots, q_{13}$ tem comprimento $n + 1$. Como n é no mínimo p, sabemos que $n + 1$ é maior que p, o número de estados de M. Assim, a seqüência tem de conter um estado repetido. Esse resultado é um exemplo do ***princípio da casa de pombos***, um nome pomposo para o fato um tanto óbvio que se p pombos forem colocados em menos que p casas, alguma casa recebe mais de um pombo.

A figura a seguir mostra a cadeia s e a seqüência de estados pelos quais M passa quando processa s. O estado q_9 é o que se repete.

$$s = s_1 \; s_2 \; s_3 \; s_4 \; s_5 \; s_6 \; \cdots \; s_n$$

$$q_1 \quad q_3 \quad q_{20} \quad q_9 \quad q_{17} \quad q_9 \quad q_6 \qquad q_{35} \quad q_{13}$$

FIGURA 1.71
Exemplo mostrando o estado q_9 se repetindo quando M lê s.

Agora dividimos s nas três partes x, y e z. A parte x é a parte de s que aparece antes de q_9, a parte y é a que ocorre entre as duas aparições de q_9, e a parte z é o resto de s, vindo após a segunda ocorrência de q_9. Portanto, x leva M do estado q_1 para q_9, y leva M de q_9 de volta para q_9 e z leva M de q_9 para o estado de aceitação q_{13}, como mostrado na figura a seguir.

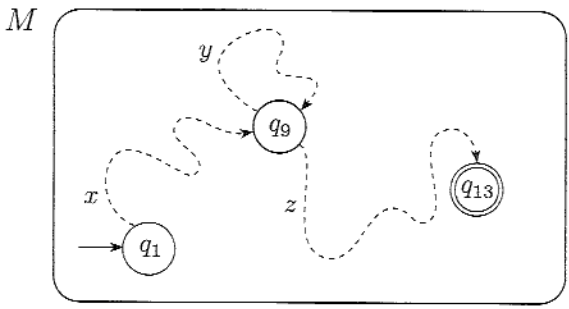


FIGURA 1.72
Exemplo mostrando como as cadeias x, y e z afetam M.

Vamos ver por que essa divisão de s satisfaz as três condições. Suponha que rodemos M sobre a entrada $xyyz$. Sabemos que x leva M de q_1 para q_9, e então o primeiro y o leva de q_9 de volta para q_9, assim como faz o segundo y, e depois z o leva para q_{13}. Com q_{13} sendo um estado de aceitação, M aceita a entrada $xyyz$. Analogamente, ele irá aceitar xy^iz para qualquer $i > 0$. Para o caso $i = 0$, $xy^iz = xz$, que é aceita por razões semelhantes. Isso estabelece a condição 1.

Verificando a condição 2, vemos que $|y| > 0$, pois y era a parte de s que aparecia entre duas ocorrências diferentes do estado q_9.

Para assegurar a condição 3, façamos com que q_9 seja a primeira repetição na seqüência. Pelo princípio da casa de pombos, os primeiros $p + 1$ estados na seqüência devem conter uma repetição. Por conseguinte, $|xy| \leq p$.

PROVA Seja $M = (Q, \Sigma, \delta, q_1, F)$ um AFD que reconhece A e p o número de estados de M.

Seja $s = s_1 s_2 \cdots s_n$ uma cadeia em A de comprimento n, onde $n \geq p$. Seja $r_1, \ldots, r_{n+1}$ a seqüência de estados nos quais M passa enquanto processa s, de forma que $r_{i+1} = \delta(r_i, s_i)$ para $1 \leq i \leq n$. Essa seqüência tem comprimento

$n+1$, que é pelo menos $p+1$. Entre os primeiros $p+1$ elementos da seqüência, dois devem ser o mesmo estado, pelo princípio da casa de pombos. Chamamos o primeiro desses de r_j e o segundo de r_l. Como r_l ocorre entre as primeiras $p+1$ posições da seqüência começando em r_1, temos que $l \leq p+1$. Agora, seja $x = s_1 \cdots s_{j-1}$, $y = s_j \cdots s_{l-1}$ e $z = s_l \cdots s_n$.

Como x leva M de r_1 para r_j, y leva M de r_j para r_j e z leva M de r_j para r_{n+1}, que é um estado de aceitação, M deve aceitar xy^iz para $i \geq 0$. Sabemos que $j \neq l$, e portanto $|y| > 0$; e $l \leq p+1$, e logo $|xy| \leq p$. Dessa forma, satisfizemos todas as condições do lema do bombeamento.

Para usar o lema do bombeamento para provar que uma linguagem B não é regular, primeiro suponha que B seja regular, a fim de obter uma contradição. Então use o lema do bombeamento para garantir a existência de um comprimento de bombeamento p de forma que todas as cadeias de comprimento p ou maiores em B possam ser bombeadas. Em seguida, encontre uma cadeia s em B que tenha comprimento p ou mais, mas que não possa ser bombeada. Finalmente, demonstre que s não pode ser bombeada considerando todas as maneiras de dividir s em x, y e z (levando a condição 3 do lema do bombeamento em consideração se for conveniente) e, para cada divisão, encontrando um valor i tal que $xy^iz \notin B$. Esse passo final freqüentemente envolve agrupar as várias formas de se dividir s em vários casos e analisá-los individualmente. A existência de s contradiz o lema do bombeamento se B for regular. Assim, B não pode ser regular.

Encontrar s algumas vezes requer um pouco de pensamento criativo. Você pode ter que tentar vários candidatos para s antes de descobrir um que funcione. Tente membros de B que pareçam exibir a "essência" da não-regularidade de B. Discutiremos mais a tarefa de encontrar s em alguns dos exemplos a seguir.

EXEMPLO 1.73

Seja B a linguagem $\{0^n1^n | n \geq 0\}$. Usamos o lema do bombeamento para provar que B não é regular. A prova é por contradição.

Suponha, ao contrário, que B seja regular. Seja p o comprimento de bombeamento dado pelo lema do bombeamento. Escolha s como a cadeia 0^p1^p. Como s é um membro de B e tem comprimento maior que p, o lema do bombeamento garante que s pode ser dividida em três partes, $s = xyz$, onde para qualquer $i \geq 0$ a cadeia xy^iz está em B. Consideramos três casos para mostrar que esse resultado é impossível.

1. A cadeia y contém apenas 0s. Neste caso, a cadeia $xyyz$ tem mais 0s que 1s e, portanto, não é um membro de B, violando a condição 1 do lema do bombeamento. Esse caso é uma contradição.
2. A cadeia y contém somente 1s. Esse caso também dá uma contradição.
3. A cadeia y contém ambos, 0s e 1s. Nesse caso, a cadeia $xyyz$ pode ter o

mesmo número de 0s e 1s, mas eles estarão fora de ordem, com alguns 1s antes de 0s. Logo, ela não é um membro de B, o que é uma contradição.

Assim, uma contradição é inevitável se fazemos a suposição de que B é regular; portanto, B não é regular. Note que podemos simplificar esse argumento aplicando a condição 3 do lema do bombeamento para eliminar os casos 2 e 3.

Nesse exemplo, encontrar a cadeia s foi fácil, porque qualquer cadeia em B de comprimento p ou mais funcionaria. Nos próximos dois exemplos algumas escolhas para s não funcionam; portanto, um cuidado adicional é necessário. ■

EXEMPLO 1.74

Seja $C = \{w \mid w$ tem número igual de 0s e 1s$\}$. Usamos o lema do bombeamento para provar que C não é regular. A prova é por contradição.

Suponha, ao contrário, que C seja regular. Seja p o comprimento de bombeamento dado pelo lema do bombeamento. Como no Exemplo 1.73, seja s a cadeia 0^p1^p. Com s sendo um membro de C e tendo comprimento maior que p, o lema do bombeamento garante que s pode ser dividida em três partes, $s = xyz$, onde para qualquer $i \geq 0$ a cadeia xy^iz está em C. Gostaríamos de mostrar que isso é impossível. Mas espere, isso *é* possível! Se fizermos x e z serem a cadeia vazia e y ser a cadeia 0^p1^p, então xy^iz sempre terá um número igual de 0s e 1s e, portanto, está em C. Logo, *parece* que s pode ser bombeada.

Aqui a condição 3 no lema do bombeamento é útil. Ela estipula que, quando bombeando s, ela deve ser dividida de forma que $|xy| \leq p$. Essa restrição na maneira em que s pode ser dividida torna mais fácil mostrar que a cadeia $s = 0^p1^p$ que selecionamos não pode ser bombeada. Se $|xy| \leq p$, então y deve conter somente 0s; logo, $xyyz \notin C$. Por conseguinte, s não pode ser bombeada. O que nos dá a contradição desejada.[7]

A seleção da cadeia s nesse exemplo exigiu mais cuidado do que no Exemplo 1.73. Se tivéssemos escolhido $s = (01)^p$, teríamos tido problema, pois precisamos de uma cadeia que *não possa* ser bombeada e essa cadeia *pode* ser bombeada, mesmo levando em conta a condição 3. Você pode ver como bombeá-la? Uma maneira seria mediante as atribuições $x = \varepsilon$, $y = 01$ e $z = (01)^{p-1}$. Com isso, $xy^iz \in C$ para todo valor de i. Se você falhar na sua primeira tentativa de encontrar uma cadeia que não possa ser bombeada, não se desespere. Tente outra!

Um método alternativo de provar que C é não-regular segue de nosso conhecimento que B é não-regular. Se C fosse regular, $C \cap 0^*1^*$ também seria regular. Os motivos são que a linguagem 0^*1^* é regular e que a classe das linguagens regulares é fechada sob interseção, resultado que provamos na nota de rodapé 3 (página 47). Mas $C \cap 0^*1^*$ é igual a B, e sabemos que B é não-regular do Exemplo 1.73. ■

[7] Poderíamos ter usado a condição 3 no Exemplo 1.73 também, para simplificar sua prova.

EXEMPLO 1.75

Seja $F = \{ww \mid w \in \{0,1\}^*\}$. Mostramos que F é não-regular usando o lema do bombeamento.

Suponha, ao contrário, que F seja regular. Seja p o comprimento de bombeamento dado pelo lema do bombeamento. Seja s a cadeia 0^p10^p1. Como s é um membro de F e s tem comprimento maior que p, o lema do bombeamento garante que s pode ser dividida em três partes, $s = xyz$, satisfazendo as três condições do lema. Mostramos que isso é impossível.

A condição 3 é outra vez crucial, pois sem ela poderíamos bombear s se fizéssemos x e z iguais à cadeia vazia. Com a condição 3, a prova se concretiza, visto que y pode conter apenas 0s; logo, $xyyz \notin F$.

Observe que escolhemos $s = 0^p10^p1$ como uma cadeia que exibe a "essência" da não-regularidade de F, em vez de, digamos, a cadeia 0^p0^p. Muito embora 0^p0^p seja um membro de F, ela não serve para demonstrar uma contradição, pois pode ser bombeada. ■

EXEMPLO 1.76

Aqui demonstramos a não regularidade de uma linguagem unária. Seja $D = \{1^{n^2} \mid n \geq 0\}$. Em outras palavras, D contém todas as cadeias de 1s cujo comprimento é um quadrado perfeito. Usamos o lema do bombeamento para provar que D não é regular. A prova é por contradição.

Suponha, ao contrário, que D seja regular. Seja p o comprimento de bombeamento dado pelo lema do bombeamento. Seja s a cadeia 1^{p^2}. Como s é um membro de D e s tem comprimento no mínimo p, o lema do bombeamento garante que s pode ser dividido em três partes, $s = xyz$, onde para qualquer $i \geq 0$ a cadeia xy^iz está em D. Como nos exemplos precedentes, mostramos que isso é impossível. Fazer isso nesse caso requer um pouco de raciocínio sobre a seqüência de quadrados perfeitos:

$$0, 1, 4, 9, 16, 25, 36, 49, \ldots$$

Note a crescente distância entre membros sucessivos dessa seqüência. Membros grandes dessa seqüência não podem estar próximos um do outro.

Agora considere as duas cadeias xyz e xy^2z. Essas cadeias diferem uma da outra por uma única repetição de y, e conseqüentemente seus comprimentos diferem entre si do comprimento de y. Pela condição 3 do lema do bombeamento, $|xy| \leq p$ e, assim, $|y| \leq p$. Temos que $|xyz| = p^2$ e, então, $|xy^2z| \leq p^2 + p$. Mas $p^2 + p < p^2 + 2p + 1 = (p+1)^2$. Além disso, a condição 2 implica que y não é a cadeia vazia, e, assim, $|xy^2z| > p^2$. Portanto, o comprimento de xy^2z fica estritamente entre os quadrados perfeitos consecutivos p^2 e $(p+1)^2$. Conseqüentemente, ele não pode ser um quadrado perfeito. Dessa forma, chegamos à contradição $xy^2z \notin D$ e concluímos que D não é regular. ■

EXEMPLO 1.77

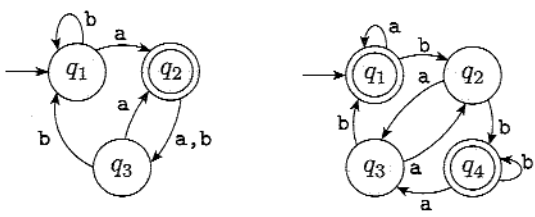
Algumas vezes, "bombear para baixo" é útil quando aplicamos o lema do bombeamento. Usamos o lema do bombeamento para mostrar que $E = \{0^i 1^j | i > j\}$ não é regular. A prova é por contradição.

Suponha que E seja regular. Seja p o comprimento de bombeamento para E dado pelo lema do bombeamento. Seja $s = 0^{p+1}1^p$. Então s pode ser dividida em xyz, satisfazendo as condições do lema do bombeamento. Pela condição 3, y contém somente 0s. Vamos examinar a cadeia $xyyz$ para ver se ela pode estar em E. Adicionar uma cópia extra de y aumenta o número de 0s. Mas, como E contém todas as cadeias em 0^*1^* que têm mais 0s que 1s, aumentando-se o número de 0s obtém-se ainda uma cadeia em E. Não ocorre contradição. Precisamos tentar alguma outra coisa.

O lema do bombeamento afirma que $xy^iz \in E$ mesmo quando $i = 0$; assim, vamos considerar a cadeia $xy^0z = xz$. Removendo-se a cadeia y, o número de 0s em s diminui. Lembre-se que s tem apenas um 0 a mais que 1s. Portanto, xz não pode ter mais 0s que 1s e, logo, não pode ser um membro de E. Dessa forma, obtemos uma contradição. ■

EXERCÍCIOS

[R]**1.1** A seguir estão os diagramas de estado de dois AFDs, M_1 e M_2. Responda às seguintes questões sobre cada uma dessas máquinas.



M_1 M_2

a. Qual é o estado inicial?

b. Qual é o conjunto de estados de aceitação?

c. Por qual seqüência de estados a máquina passa para a entrada `aabb`?

d. A máquina aceita a cadeia `aabb`?

e. A máquina aceita a cadeia ε?

[R]**1.2** Dê a descrição formal das máquinas M_1 e M_2 desenhadas no Exercício 1.1.

1.3 A descrição formal de um AFD M é $\big(\{q_1, q_2, q_3, q_4, q_5\}, \{\mathtt{u}, \mathtt{d}\}, \delta, q_3, \{q_3\}\big)$, onde δ é dada pela tabela a seguir. Dê o diagrama de estados dessa máquina.

	u	d
q_1	q_1	q_2
q_2	q_1	q_3
q_3	q_2	q_4
q_4	q_3	q_5
q_5	q_4	q_5

1.4 Cada uma das linguagens a seguir é a interseção de duas linguagens mais simples. Em cada caso, construa AFDs para as linguagens mais simples, e depois combine-os usando a construção discutida na nota de rodapé 3 (página 47) para obter o diagrama de estados de um AFD para a linguagem dada. Em todos as casos, $\Sigma = \{\mathtt{a}, \mathtt{b}\}$.

a. $\{w \mid w$ tem pelo menos três as e pelo menos dois bs$\}$

[R]**b.** $\{w \mid w$ tem exatamente dois as e pelo menos dois bs$\}$

c. $\{w \mid w$ tem um número par de as e um ou dois bs$\}$

[R]**d.** $\{w \mid w$ tem um número par de as e cada a é seguido por pelo menos um b$\}$

e. $\{w \mid w$ tem um número par de as e um ou dois bs$\}$

f. $\{w \mid w$ tem um número ímpar de as e termina com um b$\}$

g. $\{w \mid w$ tem comprimento par e um número ímpar de as$\}$

1.5 Cada uma das linguagens a seguir é o complemento de uma linguagem mais simples. Em cada caso, construa um AFD para a linguagem mais simples, e use-o para obter o diagrama de estados de um AFD para a linguagem dada. Em todos os casos $\Sigma = \{\mathtt{a}, \mathtt{b}\}$.

[R]**a.** $\{w \mid w$ não contém a subcadeia ab$\}$

[R]**b.** $\{w \mid w$ não contém a subcadeia baba$\}$

c. $\{w \mid w$ não contém nem a subcadeia ab, nem ba$\}$

d. $\{w \mid w$ é qualquer cadeia que não está em $\mathtt{a}^*\mathtt{b}^*\}$

e. $\{w \mid w$ é qualquer cadeia que não está em $(\mathtt{ab}^+)^*\}$

f. $\{w \mid w$ é qualquer cadeia que não está em $\mathtt{a}^* \cup \mathtt{b}^*\}$

g. $\{w \mid w$ é qualquer cadeia que não contém exatamente dois as$\}$

h. $\{w \mid w$ é qualquer cadeia, exceto a e b$\}$

1.6 Dê diagramas de estado de AFDs que reconhecem as linguagens a seguir. Em todos os casos o alfabeto é $\{0,1\}$

a. $\{w \mid w$ começa com um 1 e termina com um 0$\}$

b. $\{w \mid w$ contém pelo menos três 1s$\}$

c. $\{w \mid w$ contém a subcadeia 0101, isto é, $w = x0101y$ para algum x e algum $y\}$

d. $\{w \mid w$ tem comprimento pelo menos 3 e seu terceiro símbolo é um 0$\}$

e. $\{w \mid w$ começa com 0 e tem comprimento ímpar, ou começa com 1 e tem comprimento par$\}$

f. $\{w \mid w$ não contém a subcadeia 110$\}$
g. $\{w \mid$ o comprimento de w é no máximo 5$\}$
h. $\{w \mid w$ é qualquer subcadeia exceto 11 e 111$\}$
i. $\{w \mid$ toda posição ímpar de w é um 1$\}$
j. $\{w \mid w$ contém pelo menos dois 0s e no máximo um 1$\}$
k. $\{\varepsilon, 0\}$
l. $\{w \mid w$ contém um número par de 0s, ou contém exatamente dois 1s$\}$
m. O conjunto vazio
n. Todas as cadeias exceto a cadeia vazia

1.7 Dê diagramas de estado de AFNs com o número especificado de estados reconhecendo cada uma das linguagens a seguir. Em todos os casos o alfabeto é $\{0,1\}$.

[R]**a.** A linguagem $\{w \mid w$ termina com 00$\}$ com três estados
b. A linguagem do Exercício 1.6c com cinco estados
c. A linguagem do Exercício 1.6l com seis estados
d. A linguagem $\{0\}$ com dois estados
e. A linguagem $0^*1^*0^+$ com três estados
[R]**f.** A linguagem $1^*(001^+)^*$ com três estados
g. A linguagem $\{\varepsilon\}$ com um estado
h. A linguagem 0^* com um estado

1.8 Use a construção dada na prova do Teorema 1.45 para apresentar os diagramas de estados dos AFNs que reconhecem as uniões das linguagens descritas em

a. Exercícios 1.6a e 1.6b.
b. Exercícios 1.6c e 1.6f.

1.9 Use a construção fornecida na prova do Teorema 1.47 para dar os diagramas de estados dos AFNs que reconhecem as concatenações das linguagens descritas em

a. Exercícios 1.6g e 1.6i.
b. Exercícios 1.6b e 1.6m.

1.10 Use a construção dada na prova do Teorema 1.49 para apresentar os diagramas de estado dos AFNs que reconhecem as estrelas das linguagens descritas em

a. Exercício 1.6b.
b. Exercício 1.6j.
c. Exercício 1.6m.

[#]**1.11** Prove que todo AFN pode ser convertido para um outro equivalente que tem um único estado de aceitação.

1.12 Seja $D = \{w \mid w$ contém um número par de as e um número ímpar de bs e não contém a subcadeia ab$\}$. Apresente um AFD com cinco estados que reconheça D e uma expressão regular que gere D. (Sugestão: descreva D com mais simplicidade.)

1.13 Seja F a linguagem de todas as cadeias sobre $\{0,1\}$ que não contenham um par de 1s que estejam separados por um número ímpar de símbolos. Apresente o diagrama de estados de um AFD com cinco estados que reconheça F. (Você pode achar útil encontrar primeiro um AFN de quatro estados para o complemento de F.)

1.14 **a.** Mostre que, se M é um AFD que reconhece a linguagem B, tornando-se estados de aceitação os estados de M que não são de aceitação, e vice-versa, obtém-se um novo AFD que reconhece o complemento de B. Conclua que a classe das linguagens regulares é fechada sob complemento.

b. Mostre por meio de um exemplo que, se M é um AFN que reconhece a linguagem C, tornando-se estados de aceitação os estados de M que não são de aceitação, e vice-versa, não necessariamente se obtém um novo AFN que reconhece o complemento de C. A classe das linguagens reconhecidas por AFNs é fechada sob complemento? Explique sua resposta.

1.15 Dê um contra-exemplo para mostrar que a seguinte construção falha em provar o Teorema 1.49, o fecho da classe de linguagens regulares sob a operação estrela.[8] Suponha que $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ reconheça A_1. Construa $N = (Q_1, \Sigma, \delta, q_1, F)$ como segue. N supostamente reconhece A_1^*.

a. Os estados de N são os estados de N_1.

b. O estado inicial de N é o mesmo que o estado incial de N_1.

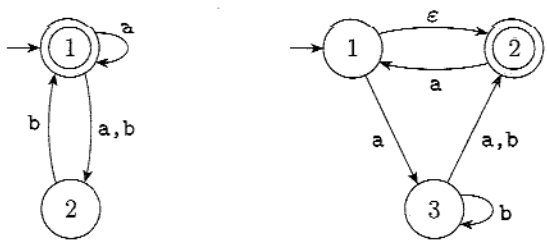
c. $F = \{q_1\} \cup F_1$.
Os estados de aceitação F são os estados de aceitação antigos mais seu estado inicial.

d. Defina δ de modo que para qualquer $q \in Q$ e qualquer $a \in \Sigma_\varepsilon$,

$$\delta(q,a) = \begin{cases} \delta_1(q,a) & q \notin F_1 \text{ ou } a \neq \varepsilon \\ \delta_1(q,a) \cup \{q_1\} & q \in F_1 \text{ e } a = \varepsilon. \end{cases}$$

(Sugestão: Mostre essa construção graficamente, como na Figura 1.50.)

1.16 Use a construção dada no Teorema 1.39 para converter os dois autômatos finitos não-determinísticos apresentados a seguir em autômatos finitos determinísticos equivalentes.



(a) (b)

1.17 **a.** Dê um AFN que reconheça a linguagem $(01 \cup 001 \cup 010)^*$.

b. Converta esse AFN em um AFD equivalente. Apresente apenas a parte do AFD que é alcançável a partir do estado inicial.

1.18 Dê expressões regulares que gerem as linguagens do Exercício 1.6.

---

[8] Em outras palavras, você deve apresentar um autômato finito, N_1, para o qual o autômato construído, N, não reconhece a estrela da linguagem de N_1.

1.19 Use o procedimento descrito no Lema 1.55 para converter as seguintes expressões regulares em autômatos finitos não-determinísticos.

a. $(0 \cup 1)^*000(0 \cup 1)^*$

b. $(((00)^*(11)) \cup 01)^*$

c. $\emptyset^*$

1.20 Para cada uma das linguagens a seguir, apresente duas cadeias que sejam membros e duas que *não* sejam membros — um total de quatro cadeias para cada caso. Presuponha o alfabeto $\Sigma = \{a,b\}$ em todos os casos.

a. a^*b^*

b. $a(ba)^*b$

c. $a^* \cup b^*$

d. $(aaa)^*$

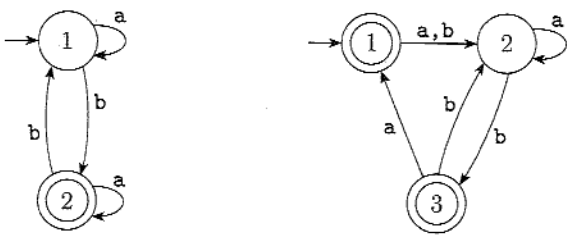
e. $\Sigma^*a\Sigma^*b\Sigma^*a\Sigma^*$

f. $aba \cup bab$

g. $(\varepsilon \cup a)b$

h. $(a \cup ba \cup bb)\Sigma^*$

1.21 Use o procedimento descrito no Lema 1.60 para converter os seguintes autômatos finitos em expressões regulares.



(a) (b)

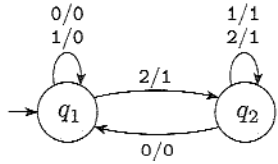
1.22 Em certas linguagens de programação, comentários ocorrem entre delimitadores como /# e #/. Seja C a linguagem de todas as cadeias de comentários delimitados. Um membro de C deve começar com /# e terminar com #/, mas não pode ter qualquer outra ocorrência de #/. Por simplicidade, vamos dizer que os comentários propriamente ditos são escritos com apenas os símbolos a e b; assim, o alfabeto de C é $\Sigma = \{a, b, /, \#\}$.

a. Dê um AFD que reconheça C.

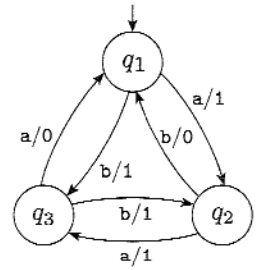
b. Dê uma expressão regular que gere C.

R1.23 Seja B qualquer linguagem sobre o alfabeto Σ. Prove que $B = B^+$ sse $BB \subseteq B$.

1.24 Um ***transdutor de estado finito*** (TEF) é um tipo de autômato finito determinístico cuja saída é uma cadeia, e não simplesmente *aceite* ou *rejeite*. Os diagramas de estado a seguir são dos transdutores de estado finito T_1 e T_2.



T_1



T_2

Cada transição de um TEF é rotulada com dois símbolos, um designando o símbolo de entrada para aquela transição, e outro designando o símbolo de saída. Os dois símbolos são escritos com uma barra, /, separando-os. Em T_1, a transição de q_1 para q_2 tem símbolos de entrada 2 e símbolo de saída 1. Algumas transições podem ter múltiplos pares de entrada–saída, como a transição em T_1 de q_1 para si próprio. Quando um TEF computa sobre uma cadeia de entrada w, ele toma os símbolos de entrada $w_1 \cdots w_n$ um por um e, começando no estado inicial, segue as transições respeitando a igualdade entre os rótulos de entrada e a seqüência de símbolos $w_1 \cdots w_n = w$. Toda vez que ele passa por uma transição, ele dá como saída o símbolo de saída correspondente. Por exemplo, sobre a entrada 2212011, a máquina T_1 passa pela seqüência de estados $q_1, q_2, q_2, q_2, q_2, q_1, q_1, q_1$ e produz a saída 1111000. Sobre a entrada abbb, T_2 produz a saída 1011. Dê a seqüência de estados visitados e a saída produzida em cada um dos seguintes casos.

a. T_1 sobre a entrada 011
b. T_1 sobre a entrada 211
c. T_1 sobre a entrada 121
d. T_1 sobre a entrada 0202
e. T_2 sobre a entrada b
f. T_2 sobre a entrada bbab
g. T_2 sobre a entrada bbbbbb
h. T_2 sobre a entrada ε

1.25 Leia a definição informal do transdutor de estado finito dada no Exercício 1.24. Dê uma definição formal desse modelo, seguindo o padrão da Definição 1.5 (página 36). Suponha que um TEF tem um alfabeto de entrada Σ e um alfabeto de saída Γ mas não um conjunto de estados de aceitação. Inclua uma definição formal da computação de um TEF. (Dica: Um TEF é uma 5-upla. Sua função de transição é da forma $\delta\colon Q \times \Sigma \longrightarrow Q \times \Gamma$.)

1.26 Usando a solução que você forneceu para o Exercício 1.25, dê uma descrição formal das máquinas T_1 e T_2 apresentadas no Exercício 1.24.

1.27 Leia a definição informal do transdutor de estado finito dada no Exercício 1.24. Dê o diagrama de estados de um TEF com o seguinte comportamento. Seus alfabetos de entrada e de saída são {0,1}. Sua cadeia de saída é idêntica à de entrada nas posições pares, mas invertida nas posições ímpares. Por exemplo, sobre a entrada 0000111 ele deve emitir a saída 1010010.

1.28 Converta as seguintes expressões regulares em AFNs usando o procedimento dado no Teorema 1.54. Em todos os casos $\Sigma = \{\mathtt{a}, \mathtt{b}\}$.

a. $\mathtt{a}(\mathtt{abb})^* \cup \mathtt{b}$

b. $\mathtt{a}^+ \cup (\mathtt{ab})^+$

c. $(\mathtt{a} \cup \mathtt{b}^+)\mathtt{a}^+\mathtt{b}^+$

1.29 Use o lema do bombeamento para mostrar que as linguagens a seguir não são regulares.

R**a.** $A_1 = \{0^n 1^n 2^n \mid n \geq 0\}$

b. $A_2 = \{www \mid w \in \{\mathtt{a}, \mathtt{b}\}^*\}$

R**c.** $A_3 = \{\mathtt{a}^{2^n} \mid n \geq 0\}$ (Aqui, $\mathtt{a}^{2^n}$ significa uma cadeia de 2^n as.)

1.30 Descreva o erro na seguinte "prova" de que 0^*1^* não é uma linguagem regular. (Deve existir um erro, pois 0^*1^* *é* regular.) A prova é por contradição. Suponha que 0^*1^* seja regular. Seja p o comprimento de bombeamento para 0^*1^* dado pelo lema do bombeamento. Escolha s como a cadeia 0^p1^p. Você sabe que s é um membro de 0^*1^*, mas o Exemplo 1.73 mostra que s não pode ser bombeada. Assim, você tem uma contradição. Portanto, 0^*1^* não é regular.

PROBLEMAS

1.31 Para qualquer cadeia $w = w_1w_2 \cdots w_n$, o ***reverso*** de w, escrito $w^{\mathcal{R}}$, é a cadeia w na ordem reversa, $w_n \cdots w_2w_1$. Para qualquer linguagem A, seja $A^{\mathcal{R}} = \{w^{\mathcal{R}} | w \in A\}$. Mostre que se A é regular, $A^{\mathcal{R}}$ também o é.

1.32 Seja

$$\Sigma_3 = \left\{ \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \cdots, \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \right\}.$$

Σ_3 contém todas as colunas de tamanho 3 de 0s e 1s. Uma cadeia de símbolos em Σ_3 dá três linhas de 0s e 1s. Considere cada linha como um número binário e seja

$$B = \{w \in \Sigma_3^* | \text{ a última linha de } w \text{ é a soma das duas primeiras}\}.$$

Por exemplo,

$$\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} \in B, \qquad \text{mas} \qquad \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \notin B.$$

Mostre que B é regular. (Dica: trabalhar com $B^{\mathcal{R}}$ é mais fácil. Você pode supor o resultado afirmado no Problema 1.31.)

1.33 Seja

$$\Sigma_2 = \left\{ \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right\}.$$

Aqui, Σ_2 possui todas as colunas de 0s e 1s de altura dois. Uma cadeia de símbolos em Σ_2 dá duas linhas de 0s e 1s. Considere cada linha como um número binário, e seja

$$C = \{w \in \Sigma_2^* | \text{ a última linha de } w \text{ é três vezes a primeira}\}.$$

Por exemplo, $\begin{bmatrix} 0 \\ 0 \end{bmatrix}\begin{bmatrix} 0 \\ 1 \end{bmatrix}\begin{bmatrix} 1 \\ 1 \end{bmatrix}\begin{bmatrix} 0 \\ 0 \end{bmatrix} \in C$, mas $\begin{bmatrix} 0 \\ 1 \end{bmatrix}\begin{bmatrix} 0 \\ 1 \end{bmatrix}\begin{bmatrix} 1 \\ 0 \end{bmatrix} \notin C$. Mostre que C é regular. (Você pode supor o resultado afirmado no Problema 1.31.)

1.34 Seja Σ_2 o mesmo que no Problema 1.33. Considere cada linha como um número binário, e seja

$$D = \{w \in \Sigma_2^* \mid \text{o número na primeira linha de } w \text{ é maior que o número na última linha}\}.$$

Por exemplo, $\left[\begin{smallmatrix}0\\0\end{smallmatrix}\right]\left[\begin{smallmatrix}1\\0\end{smallmatrix}\right]\left[\begin{smallmatrix}1\\1\end{smallmatrix}\right]\left[\begin{smallmatrix}0\\0\end{smallmatrix}\right] \in D$, mas $\left[\begin{smallmatrix}0\\0\end{smallmatrix}\right]\left[\begin{smallmatrix}0\\1\end{smallmatrix}\right]\left[\begin{smallmatrix}1\\1\end{smallmatrix}\right]\left[\begin{smallmatrix}0\\0\end{smallmatrix}\right] \notin D$. Mostre que D é regular.

1.35 Seja Σ_2 o mesmo que no Problema 1.33. Considere a primeira e a última linhas como cadeias de 0s e 1s, e seja

$$E = \{w \in \Sigma_2^* \mid \text{a última linha de } w \text{ é o reverso da primeira linha de } w\}.$$

Mostre que E não é regular.

1.36 Seja $B_n = \{\mathtt{a}^k \mid k \text{ é um múltiplo de } n\}$. Mostre que para cada $n \geq 1$, a linguagem B_n é regular.

1.37 Seja $C_n = \{x \mid x \text{ é um número binário múltiplo de } n\}$. Mostre que para cada $n \geq 1$, a linguagem C_n é regular.

1.38 Um AFN-***de-todos-os-caminhos*** M é uma 5-upla $(Q, \Sigma, \delta, q_0, F)$ que aceita $x \in \Sigma^*$ se *todo* estado em que M pode estar após ler a entrada x é um estado de F. Note que, em contraste, um AFN ordinário aceita uma cadeia se *algum* estado entre os alcançáveis por M é um estado de aceitação. Prove que os AFNs-de-todos-os-caminhos reconhecem a classe das linguagens regulares.

1.39 A construção no Teorema 1.54 mostra que todo AFNG é equivalente a um AFNG com apenas dois estados. Podemos mostrar que um fenômeno oposto ocorre para AFDs. Prove que para todo $k > 1$ existe uma linguagem $A_k \subseteq \{0,1\}^*$ que é reconhecida por um AFD com k estados, mas não por um com apenas $k-1$ estados.

1.40 Digamos que uma cadeia x é um ***prefixo*** de uma cadeia y se existe uma cadeia z tal que $xz = y$, e que x é um ***prefixo próprio*** de y se, adicionalmente, $x \neq y$. Em cada um dos itens a seguir, definimos uma operação sobre uma linguagem A. Mostre que a classe das linguagens regulares é fechada sob aquela operação.

[R]**a.** NÃOPREFIXO$(A) = \{w \in A \mid$ nenhum prefixo próprio de w é um membro de $A\}$.

b. NÃOESTENDE$(A) = \{w \in A \mid w$ não é prefixo próprio de nenhuma cadeia em $A\}$.

1.41 Para as linguagens A e B, seja o ***embaralhamento perfeito*** de A e B a linguagem

$$\{w \mid w = a_1 b_1 \cdots a_k b_k, \text{ onde } a_1 \cdots a_k \in A \text{ e } b_1 \cdots b_k \in B, \text{ cada } a_i, b_i \in \Sigma\}.$$

Mostre que a classe das linguagens regulares é fechada sob embaralhamento perfeito.

1.42 Para as linguagens A e B, seja o ***embaralhamento*** de A e B a linguagem

$$\{w \mid w = a_1 b_1 \cdots a_k b_k, \text{ onde } a_1 \cdots a_k \in A \text{ e } b_1 \cdots b_k \in B, \text{ cada } a_i, b_i \in \Sigma^*\}.$$

Mostre que a classe das linguagens regulares é fechada sob embaralhamento.

1.43 Seja A uma linguagem qualquer. Defina $REM(A)$ como a linguagem contendo todas as cadeias que podem ser obtidas pela remoção de um símbolo de uma cadeia em A. Assim, $REM(A) = \{xz \mid xyz \in A \text{ onde } x, z \in \Sigma^*, y \in \Sigma\}$. Mostre que a classe das linguagens regulares é fechada sob a operação REM. Apresente ambas, uma prova por meio de figuras e uma prova mais formal por construção, como no Teorema 1.47.

R**1.44** Sejam B e C linguagens sobre $\Sigma = \{0, 1\}$. Defina

$$B \overset{1}{\leftarrow} C = \{w \in B \mid \text{para algum } y \in C, \text{ as cadeias } w \text{ e } y \text{ contêm número igual de } 1\text{s}\}.$$

Mostre que a classe das linguagens regulares é fechada sob a operação $\overset{1}{\leftarrow}$.

R**1.45** Seja $A/B = \{w \mid wx \in A \text{ para algum } x \in B\}$. Mostre que se A é regular e B é qualquer linguagem, então A/B é regular.

1.46 Prove que as seguintes linguagens não são regulares. Você pode usar o lema do bombeamento e o fechamento da classe das linguagens regulares sob união, interseção e complemento.

a. $\{0^n 1^m 0^n \mid m, n \geq 0\}$

R**b.** $\{0^m 1^n \mid m \neq n\}$

c. $\{w \mid w \in \{0,1\}^* \text{ não é um palíndromo}\}$[9]

d. $\{wtw \mid w, t \in \{0,1\}^+\}$

1.47 Sejam $\Sigma = \{1, \#\}$ e

$$Y = \{w \mid w = x_1\#x_2\#\cdots\#x_k \text{ para } k \geq 0, \text{ cada } x_i \in 1^*, \text{ e } x_i \neq x_j \text{ para } i \neq j\}.$$

Prove que Y não é regular.

1.48 Sejam $\Sigma = \{0,1\}$ e

$$D = \{w \mid w \text{ contém um número igual de ocorrências das subcadeias } 01 \text{ e } 10\}.$$

Logo, $101 \in D$ porque 101 contém um único 01 e um único 10, mas $1010 \notin D$ porque 1010 contém dois 10s e um 01. Mostre que D é uma linguagem regular.

1.49 **a.** Seja $B = \{1^k y \mid y \in \{0,1\}^* \text{ e } y \text{ contém pelo menos } k \text{ 1s, para } k \geq 1\}$. Mostre que B é uma linguagem regular.

b. Seja $C = \{1^k y \mid y \in \{0,1\}^* \text{ e } y \text{ contém no máximo } k \text{ 1s, para } k \geq 1\}$. Mostre que C não é uma linguagem regular.

R**1.50** Leia a definição informal de transdutor de estado finito dada no Exercício 1.24. Prove que nenhum TEF pode dar como saída $w^{\mathcal{R}}$, para toda entrada w, se os alfabetos de entrada e de saída são $\{0,1\}$.

1.51 Sejam x e y cadeias e seja L uma linguagem qualquer. Dizemos que x e y são ***distingüíveis por L*** se existe alguma cadeia z tal que exatamente uma das cadeias, xz ou yz, é um membro de L; caso contrário, para toda cadeia z, temos $xz \in L$ sempre que $yz \in L$ e dizemos que x e y são ***indistingüíveis por L***. Se x e y são indistingüíveis por L escrevemos $x \equiv_L y$. Mostre que $\equiv_L$ é uma relação de equivalência.

---

[9] Um ***palíndromo*** é uma cadeia que tem a mesma leitura da frente para trás e de trás para frente.

R*1.52 **Teorema de Myhill–Nerode.** Olhe para o Problema 1.51. Seja L uma linguagem e suponha que X seja um conjunto de cadeias. Digamos que X é ***distingüível duas-a-duas por L*** se cada duas cadeias distintas em X são distingüíveis por L. Defina o ***índice de L*** como sendo o número máximo de elementos em qualquer conjunto que é distingüível duas-a-duas por L. O índice de L pode ser finito ou infinito.

a. Mostre que, se L é reconhecida por um AFD com k estados, L tem índice no máximo k.

b. Mostre que, se o índice de L é um número finito k, ela é reconhecida por um AFD com k estados.

c. Conclua que L é regular sse ela tem um índice finito. Além disso, seu índice é o tamanho do menor AFD que a reconhece.

1.53 Seja $\Sigma = \{0, 1, +, =\}$ e

$$SOMA = \{x{=}y{+}z \mid x, y, z \text{ são inteiros binários, e } x \text{ é a soma de } y \text{ e } z\}.$$

Mostre que $SOMA$ não é regular.

1.54 Considere a linguagem $F = \{\mathtt{a}^i\mathtt{b}^j\mathtt{c}^k \mid i, j, k \geq 0 \text{ e se } i = 1 \text{ então } j = k\}$.

a. Mostre que F não é regular.

b. Mostre que F funciona como uma linguagem regular no lema do bombeamento. Em outras palavras, dê um comprimento de bombeamento p e demonstre que F satisfaz as três condições do lema do bombeamento para esse valor de p.

c. Explique porque os itens (a) e (b) não contradizem o lema do bombeamento.

1.55 O lema do bombeamento diz que toda linguagem regular tem um comprimento de bombeamento p, tal que toda cadeia na linguagem pode ser bombeada se ela tiver comprimento p ou mais. Se p é um comprimento de bombeamento para a linguagem A, o mesmo acontece com qualquer comprimento $p' \geq p$. O ***comprimento mínimo de bombeamento*** para A é o menor p que é um comprimento de bombeamento para A. Por exemplo, se $A = 01^*$, o comprimento mínimo de bombeamento é 2. A razão é que a cadeia $s = 0$ está em A, tem comprimento 1, e s não pode ser bombeada, mas qualquer cadeia em A de comprimento 2 ou mais contém um 1 e, portanto, pode ser bombeada pela divisão da mesma de forma que $x = 0$, $y = 1$, e z seja o resto. Para cada uma das linguagens a seguir, dê o comprimento mínimo de bombeamento e justique sua resposta.

R**a.** 0001^*

R**b.** 0^*1^*

c. $001 \cup 0^*1^*$

R**d.** $0^*1^+0^+1^* \cup 10^*1$

e. $(01)^*$

f. ε

g. $1^*01^*01^*$

h. $10(11^*0)^*0$

i. 1011

j. Σ^*

*1.56 Se A é um conjunto de números naturais e k um número natural maior que 1, seja

$$B_k(A) = \{w \mid w \text{ é a representação na base } k \text{ de algum número em } A\}.$$

Aqui, não permitimos 0s à esquerda na representação de um número. Por exemplo, $B_2(\{3, 5\}) = \{11, 101\}$ e $B_3(\{3, 5\}) = \{10, 12\}$. Dê um exemplo de um conjunto A para o qual $B_2(A)$ seja regular, mas $B_3(A)$ não seja. Prove que seu exemplo funciona.

1.57 Se A é uma linguagem qualquer, seja $A_{\frac{1}{2}-}$ o conjunto de todas as primeiras metades das cadeias em A, de modo que

$$A_{\frac{1}{2}-} = \{x \mid \text{ para alguma } y,\ |x| = |y| \text{ e } xy \in A\}.$$

Mostre que, se A é regular, então $A_{\frac{1}{2}-}$ também o é.

1.58 Se A é uma linguagem qualquer, seja $A_{\frac{1}{3}-\frac{1}{3}}$ o conjunto de todas as cadeias em A com suas terças partes do meio removidas, de forma que

$$A_{\frac{1}{3}-\frac{1}{3}} = \{xz \mid \text{ para alguma } y,\ |x| = |y| = |z| \text{ e } xyz \in A\}.$$

Mostre que, se A é regular, então $A_{\frac{1}{3}-\frac{1}{3}}$ não é necessariamente regular.

1.59 Seja $M = (Q, \Sigma, \delta, q_0, F)$ um AFD e seja h um estado de M denominado sua "casa". Uma ***seqüência sincronizadora*** para M e h é uma cadeia $s \in \Sigma^*$ onde $\delta(q, s) = h$ para todo $q \in Q$. (Aqui estendemos δ para cadeias, de forma que $\delta(q, s)$ é igual ao estado em que M termina quando M inicia no estado q e lê a entrada s.) Digamos que M é ***sincronizável*** se tem uma seqüência sincronizadora para algum estado h. Prove que, se M é um AFD sincronizável de k estados, então ele tem uma seqüência sincronizadora de comprimento no máximo k^3. Você pode encontrar um limite melhor que esse?

1.60 Seja $\Sigma = \{\mathtt{a}, \mathtt{b}\}$. Para cada $k \geq 1$, seja C_k a linguagem constituída de todas as cadeias que contêm um a exatamente k posições a partir da extremidade direita. Assim, $C_k = \Sigma^* \mathtt{a} \Sigma^{k-1}$. Descreva um AFN com $k + 1$ estados que reconheça C_k, ambos em termos de um diagrama de estados e de uma descrição formal.

1.61 Considere as linguagens C_k definidas no Problema 1.60. Prove que para cada k, nenhum AFD pode reconhecer C_k com menos de 2^k estados.

1.62 Seja $\Sigma = \{\mathtt{a}, \mathtt{b}\}$. Para cada $k \geq 1$, seja D_k a linguagem constituída de todas as palavras que têm pelo menos um a entre os últimos k símbolos. Logo, $D_k = \Sigma^* \mathtt{a}(\Sigma \cup \varepsilon)^{k-1}$. Descreva um AFD com no máximo $k + 1$ estados que reconheça D_k, ambos em termos de um diagrama de estados e de uma descrição formal.

1.63
a. Seja A uma linguagem regular infinita. Prove que A pode ser dividida em dois subconjuntos regulares disjuntos infinitos.

b. Sejam B e D duas linguagens. Escreva $B \Subset D$ se $B \subseteq D$ e D contém infinitamente muitas cadeias que não estão em B. Mostre que, se B e D são duas linguagens regulares e $B \Subset D$, então podemos encontrar uma linguagem regular C tal que $B \Subset C \Subset D$.

1.64 Seja N um AFN com k estados que reconhece certa linguagem A.

a. Mostre que, se A é não-vazia, A contém alguma cadeia de comprimento no máximo k.

b. Mostre, dando um exemplo, que a parte (a) não é necessariamente verdadeira se você subsituir ambos os As por $\overline{A}$.

c. Mostre que, se $\overline{A}$ é não-vazia, $\overline{A}$ contém alguma cadeia de comprimento no máximo 2^k.

d. Mostre que o limite dado na parte (c) é quase exato; isto é, para cada k, mostre a existência de um AFN reconhecendo uma linguagem A_k tal que $\overline{A_k}$ é não-vazia e as menores cadeias membros de $\overline{A_k}$ são de comprimento exponencial em k. Aproxime-se do limite apresentado em (c) tanto quanto puder.

*1.65 Prove que, para cada $n > 0$, existe uma linguagem B_n tal que

a. B_n é reconhecível por um AFN que tem n estados, e

b. se $B_n = A_1 \cup \cdots \cup A_k$, para linguagens regulares A_i, então pelo menos uma das A_i requer um AFD com quantidade exponencial de estados.

SOLUÇÕES SELECIONADAS

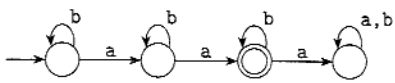
1.1 Para M_1: **(a)** q_1; **(b)** $\{q_2\}$; **(c)** q_1, q_2, q_3, q_1, q_1; **(d)** Não; **(e)** Não.
Para M_2: **(a)** q_1; **(b)** $\{q_1, q_4\}$; **(c)** q_1, q_1, q_1, q_2, q_4; **(d)** Sim; **(e)** Sim.

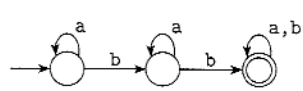
1.2 $M_2 = (\{q_1, q_2, q_3\}, \{\mathtt{a}, \mathtt{b}\}, \delta_1, q_1, \{q_2\})$.
$M_3 = (\{q_1, q_2, q_3, q_4\}, \{\mathtt{a}, \mathtt{b}\}, \delta_2, q_1, \{q_1, q_4\})$.
As funções de transição são

δ_1	a	b
q_1	q_2	q_1
q_2	q_3	q_3
q_3	q_2	q_1

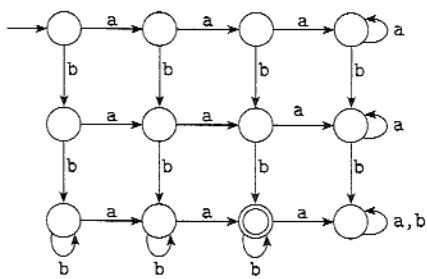
δ_2	a	b
q_1	q_1	q_2
q_2	q_3	q_4
q_3	q_2	q_1
q_4	q_3	q_4

1.4 **(b)** Seguem AFDs para as duas linguagens $\{w \mid w$ tem exatamente dois as$\}$ e $\{w \mid w$ tem pelo menos dois bs$\}$:

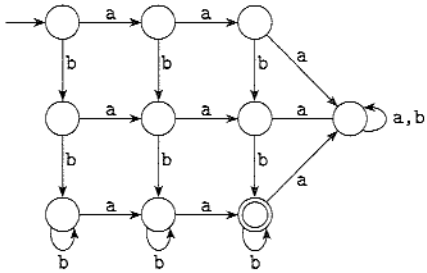




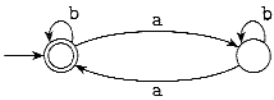
Combinando-os usando a construção para interseção, dá o AFD:

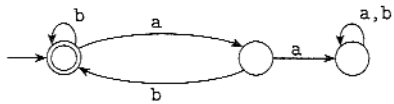


Embora o problema não lhe peça para simplificar o AFD, certos estados podem ser combinados para resultar em

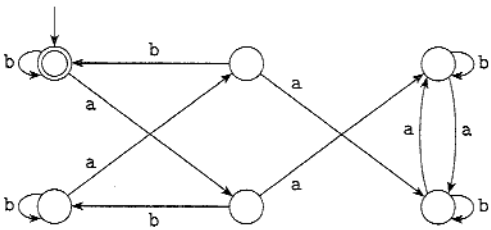


(d) Estes são AFDs para as duas linguagens $\{w \mid w$ tem um número par de as$\}$ e $\{w \mid$ cada a é seguido por pelo menos um b$\}$:

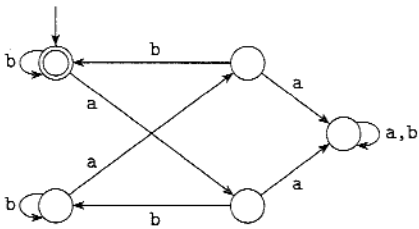




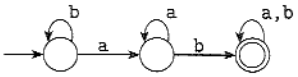
Combinando-os usando a construção para interseção, dá o AFD:

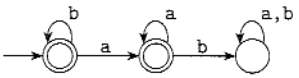


Embora o problema não lhe peça para simplificar o AFD, certos estados podem ser combinados para resultar em

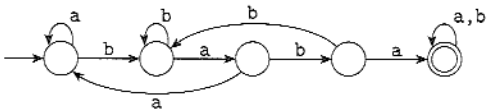


1.5 **(a)** O AFD à esquerda reconhece $\{w \mid w \text{ contém } \mathtt{ab}\}$. O AFD à direita reconhece o seu complemento, $\{w \mid w \text{ não contém } \mathtt{ab}\}$.

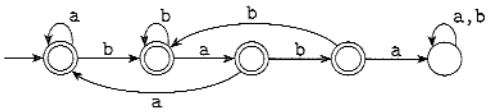




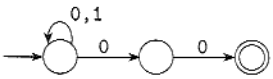
(b) Este AFD reconhece $\{w \mid w \text{ contém } \mathtt{baba}\}$.



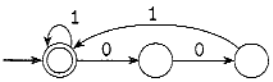
Este AFD reconhece $\{w \mid w \text{ não contém } \mathtt{baba}\}$.



1.7 **(a)**



(f)



1.11 Seja $N = (Q, \Sigma, \delta, q_0, F)$ um AFN qualquer. Construa um AFN N' com um único estado de aceitação que reconheça a mesma linguagem que N. Informalmente, N' é exatamente como N, exceto que ele tem transições-ε dos estados correspondentes aos estados de aceitação de N a um novo estado de aceitação, q_{aceita}. Não há transições saindo do estado q_{aceita}. Mais formalmente, $N' = (Q \cup \{q_{\text{aceita}}\}, \Sigma, \delta', q_0, \{q_{\text{aceita}}\})$, onde para cada $q \in Q$ e $a \in \Sigma$

$$\delta'(q,a) = \begin{cases} \delta(q,a) & \text{if } a \neq \varepsilon \text{ ou } q \notin F \\ \delta(q,a) \cup \{q_{\text{aceita}}\} & \text{if } a = \varepsilon \text{ e } q \in F \end{cases}$$

e $\delta'(q_{\text{aceita}}, a) = \emptyset$ para cada $a \in \Sigma_\varepsilon$.

1.23 Provamos ambas as direções do "sse".
($\rightarrow$) Suponha que $B = B^+$ e mostre que $BB \subseteq B$.
Para toda linguagem $BB \subseteq B^+$ se verifica; logo, se $B = B^+$, então $BB \subseteq B$.
($\leftarrow$) Suponha que $BB \subseteq B$ e mostre que $B = B^+$.
Para toda linguagem $B \subseteq B^+$; logo, temos de mostrar apenas que $B^+ \subseteq B$. Se $w \in B^+$, então $w = x_1 x_2 \cdots x_k$ onde cada $x_i \in B$ e $k \geq 1$. Como $x_1, x_2 \in B$ e $BB \subseteq B$, temos que $x_1 x_2 \in B$. Similarmente, como $x_1 x_2$ está em B e x_3 está em B, temos que $x_1 x_2 x_3 \in B$. Continuando dessa maneira, $x_1 \cdots x_k \in B$. Portanto, $w \in B$ e, assim, podemos concluir que $B^+ \subseteq B$.

O último argumento pode ser escrito formalmente como a seguinte prova por indução. Suponha que $BB \subseteq B$.
Afirmativa: Para cada $k \geq 1$, se $x_1, \ldots, x_k \in B$, então $x_1 \cdots x_k \in B$.
Base: Prova para $k = 1$. O enunciado é obviamente verdadeiro.
Passo da indução: Para cada $k \geq 1$, suponha que a afirmativa seja verdadeira para k e prove que ela é verdadeira para $k + 1$.
Se $x_1, \ldots, x_k, x_{k+1} \in B$, então pela hipótese de indução, $x_1 \cdots x_k \in B$. Portanto, $x_1 \cdots x_k x_{k+1} \in BB$; mas $BB \subseteq B$; logo, $x_1 \cdots x_{k+1} \in B$. Isso prova o passo de indução e a afirmativa. A afirmativa implica que, se $BB \subseteq B$, então $B^+ \subseteq B$.

1.29 **(a)** Suponha que $A_1 = \{0^n 1^n 2^n \mid n \geq 0\}$ seja regular. Seja p o comprimento de bombeamento dado pelo lema do bombeamento. Escolha s como a $0^p 1^p 2^p$. Como s é um membro de A_1 e s é maior que p, o lema do bombeamento garante que s pode ser dividida em três partes, $s = xyz$, sendo que para qualquer $i \geq 0$ a palavra xy^iz está em A_1. Considere duas possibilidades:

1. A cadeia y contém somente 0s, somente 1s, ou somente 2s. Nesses casos, a cadeia $xyyz$ não terá número igual de 0s, 1s e 2s. Portanto, $xyyz$ não é um membro de A_1, uma contradição.
2. A cadeia y contém mais de um tipo de símbolo. Nesse caso, $xyyz$ terá 0s, 1s ou 2s fora de ordem. Logo, $xyyz$ não é um membro de A_1, uma contradição.

Em ambos os casos chegamos a uma contradição. Portanto, A_1 não é regular.

(c) Suponha que $A_3 = \{\mathtt{a}^{2^n} \mid n \geq 0\}$ seja regular. Seja p o comprimento de bombeamento dado pelo lema do bombeamento. Escolha s como sendo a cadeia $\mathtt{a}^{2^p}$. Como s é um membro de A_1 e s é maior que p, o lema do bombeamento garante que s pode ser dividida em três partes, $s = xyz$, satisfazendo as três condições do lema do bombeamento.

A terceira condição nos diz que $|xy| \leq p$. Além disso, $p < 2^p$ e, portanto, $|y| < 2^p$. Conseqüentemente, $|xyyz| = |xyz| + |y| < 2^p + 2^p = 2^{p+1}$. A segunda condição requer que $|y| > 1$; portanto, $2^p < |xyyz| < 2^{p+1}$. O comprimento de $xyyz$ não pode ser uma potência de 2. Logo, $xyyz$ não é um membro de A_3, uma contradição. Conseqüentemente, A_3 não é regular.

1.40 Seja $M = (Q, \Sigma, \delta, q_0, F)$ um AFN reconhecendo A, sendo A uma linguagem regular. Construa $M' = (Q', \Sigma, \delta', q_0{}', F')$ reconhecendo NÃOPREFIXO(A) como segue:

1. $Q' = Q$.
2. Para $r \in Q'$ e $a \in \Sigma$ defina $\delta'(r, a) = \begin{cases} \delta(r, a) & \text{se } r \notin F \\ \emptyset & \text{se } r \in F. \end{cases}$
3. $q_0{}' = q_0$.
4. $F' = F$.

1.44 Seja $M_B = (Q_B, \Sigma, \delta_B, q_B, F_B)$ e $M_C = (Q_C, \Sigma, \delta_C, q_C, F_C)$ AFDs reconhecendo B e C, respectivamente. Construa o AFN $M = (Q, \Sigma, \delta, q_0, F)$ que reconhece $B \overset{1}{\leftarrow} C$ como segue. Para decidir se sua entrada w está em $B \overset{1}{\leftarrow} C$, a máquina M verifica se $w \in B$ e, em paralelo, não-deterministicamente conjectura uma cadeia y que contenha o mesmo número 1s que os contidos em w e verifica se $y \in C$.

1. $Q = Q_B \times Q_C$.

2. Para $(q, r) \in Q$ e $a \in \Sigma$ defina

$$\delta((q,r),a) = \begin{cases} \{(\delta_B(q,0), r)\} & \text{se } a = 0 \\ \{(\delta_B(q,1), \delta_C(r,1))\} & \text{se } a = 1 \\ \{(q, \delta_C(r,0))\} & \text{se } a = \varepsilon. \end{cases}$$

3. $q_0 = (q_B, q_C)$.
4. $F = F_B \times F_C$.

1.46 **(b)** Seja $B = \{0^m 1^n \mid m \neq n\}$. Observe que $\overline{B} \cap 0^* 1^* = \{0^k 1^k \mid k \geq 0\}$. Se fosse B regular, então $\overline{B}$ seria regular e, assim, também o seria $\overline{B} \cap 0^* 1^*$. Mas, já sabemos que $\{0^k 1^k \mid k \geq 0\}$ não é regular, e, portanto, B não pode ser regular.

Alternativamente, podemos provar que B não é regular usando o lema do bombeamento diretamente, embora seja mais complicado fazê-lo. Suponha que $B = \{0^m 1^n \mid m \neq n\}$ seja regular. Seja p o comprimento de bombeamento dado pelo lema do bombeamento. Observe que $p!$ é divisível por todos os inteiros de 1 até p, onde $p! = p(p-1)(p-2)\cdots 1$. A cadeia $s = 0^p 1^{p+p!} \in B$, e $|s| \geq p$. Logo, o lema do bombeamento implica que s pode ser dividida como xyz com $x = 0^a$, $y = 0^b$ e $z = 0^c 1^{p+p!}$, onde $b \geq 1$ e $a + b + c = p$. Seja s' a cadeia $xy^{i+1}z$, onde $i = p!/b$. Então $y^i = 0^{p!}$ e, assim, $y^{i+1} = 0^{b+p!}$ e, logo, $xyz = 0^{a+b+c+p!} 1^{p+p!}$. Isso dá $xyz = 0^{p+p!} 1^{p+p!} \notin B$, uma contradição.

1.50 Suponha, ao contrário, que algum TEF T dá como saída $w^{\mathcal{R}}$ sobre a entrada w. Considere as cadeias de entrada 00 e 01. Sobre a entrada 00, T deve dar como saída 00, e sobre a entrada 01, T deve dar como saída 10. Em ambos os casos, o primeiro bit da entrada é um 0, mas os primeiros bits das saídas diferem. Operar dessa maneira é impossível para um TEF porque ele produz seu primeiro bit de saída antes de ler seu segundo bit de entrada. Portanto, não pode existir tal TEF.

1.52 **(a)** Provamos essa asserção por contradição. Seja M um AFD de k estados que reconhece L. Suponha, para efeito de uma contradição, que L tem índice maior que k. Isso significa que algum conjunto X com mais que k elementos é distingüível duas-a-duas por L. Em razão de M ter k estados, o princípio da casa de pombos implica que X contém duas cadeias distintas x e y, onde $\delta(q_0, x) = \delta(q_0, y)$. Aqui $\delta(q_0, x)$ é o estado no qual M está após iniciar no estado q_0 e ler a cadeia de entrada x. Então, para qualquer cadeia $z \in \Sigma^*$, $\delta(q_0, xz) = \delta(q_0, yz)$. Conseqüentemente, ou ambas xz e yz estão em L ou nenhuma delas está em L. Mas então x e y não são distingüíveis por L, contradizendo nossa suposição de que X é distingüível duas-a-duas por L.

(b) Suponha que $X = \{s_1, \ldots, s_k\}$ seja distingüível duas-a-duas por L. Construímos um AFD $M = (Q, \Sigma, \delta, q_0, F)$ com k estados reconhecendo L. Seja $Q = \{q_1, \ldots, q_k\}$, e defina $\delta(q_i, a)$ como sendo q_j, onde $s_j \equiv_L s_i a$ (a relação $\equiv_L$ é definida no Problema 1.51). Note que $s_j \equiv_L s_i a$ para alguma $s_j \in X$; caso contrário, $X \cup s_i a$ teria $k+1$ elementos e seria distingüível duas-a-duas por L, o que contradiria a suposição de que L tem índice k. Seja $F = \{q_i \mid s_i \in L\}$. Suponha que o estado inicial q_0 seja o q_i tal que $s_i \equiv_L \varepsilon$. M é construído de modo que, para qualquer estado q_i, $\{s \mid \delta(q_0, s) = q_i\} = \{s \mid s \equiv_L s_i\}$. Logo, M reconhece L.

(c) Suponha que L seja regular e suponha que k seja o número de estados em um AFD que reconhece L. Então do item (a) L tem índice no máximo k. Reciprocamente, se L tem índice k, então pelo item (b) ela é reconhecida por um AFD com k estados e, portanto, é regular. Para mostrar que o índice de L é o tamanho do

menor AFD que a aceita, suponha que o índice de L é *exatamente* k. Então, pelo item (b), existe um AFD com k-estados que aceita L. Esse é o tal menor AFD porque se fosse algo menor, então poderíamos mostrar pelo item (a) que o índice de L é menor que k.

L55 **(a)** O menor comprimento de bombeamento é 4. A cadeia 000 está na linguagem mas não pode ser bombeada, assim, 3 não é um comprimento de bombeamento para essa linguagem. Se s tem comprimento 4 ou mais, ela contém 1s. Dividindo s em xyz, onde x é 000, y é o primeiro 1 e z é o restante, satisfazemos as três condições do lema do bombeamento.

(b) O comprimento de bombeamento mínimo é 1. O comprimento de bombeamento não pode ser 0 porque a cadeia ε está na linguagem e não pode ser bombeada. Toda cadeia não vazia na linguagem pode ser dividida em xyz, onde $x = \varepsilon$, y é o primeiro símbolo e z é o resto. Essa divisão satisfaz as três condições.

(d) O menor comprimento de bombeamento é 3. O comprimento de bombeamento não pode ser 2 porque a cadeia 11 está na linguagem e não pode ser bombeada. Seja s uma cadeia na linguagem de comprimento no mínimo 3. Se s é gerada por $0^*1^+0^+1^*$, podemos escrevê-la como xyz, onde x é a cadeia vazia, y é o primeiro símbolo de s e z é o resto de s. Partindo s dessa maneira mostra que ela não pode ser bombeada. Se s é gerada por 10^*1, podemos escrevê-la como xyz, onde $x = 1$, $y = 0$ e z é o resto de s. Essa divisão dá uma maneira de bombear s.